Rust for C++ Programmers

A course in thirteen modules

Contents

About this book 1

Module 1 — The mental model shift 4

Module 2 — Ownership and moves 10

Module 3 — Borrowing and lifetimes 17

Module 4 — Structs, enums, and pattern matching 25

Module 5 — Error handling 32

Module 6 — Traits and generics 38

Module 7 — Smart pointers and interior mutability 45

Module 8 — Iterators, closures, and collections 52

Module 9 — Crates, modules, and Cargo 59

Module 10 — Concurrency 66

Module 11 — Async Rust 73

Module 12 — Unsafe Rust and C++ interop 79

Module 13 — Macros 87

Appendix A — Capstone projects 93

Appendix B — C++ → Rust cheat sheet 95

Appendix C — Resources 97

About this book

This book has thirteen modules and a set of capstone projects. Each module reads straight through — concepts and side-by-side C++/Rust code, no keyboard needed — and ends with optional exercises for when you're back at a machine.

The course skips what transfers unchanged (control flow, integer types, stack vs heap, cache behavior) and spends its time where the languages diverge: ownership, the borrow checker, traits, and the type-level encoding of thread safety.

Suggested pacing

Week	Modules	Milestone
1	1–3	Write code the borrow checker accepts on the first try about half the time
2	4–5	Model a domain with enums; no `unwrap()` in non-test code
3	6–7	Read and write generic code with trait bounds; know when to reach for `Rc/Arc`
4	8–9	Iterator-heavy code; a multi-crate Cargo workspace
5	10–11	A threaded and an async program, both race-free by construction
6	12–13	Call C++ from Rust and back; write one declarative macro
7+	Capstone	One project from the capstone list

Toolchain setup

1. Install via rustup: `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`. It manages compilers the way you might use multiple GCC/Clang installs, but per project via `rust-toolchain.toml`.
2. Add components: `rustup component add clippy rustfmt rust-src`.
3. Editor: rust-analyzer (VS Code, Neovim, CLion/RustRover). Turn on inlay hints for types and lifetimes — they show what the compiler infers, which is most of the learning in weeks 1–3.
4. Create a scratch workspace: `cargo new --lib rust-course && cd rust-course`. Put each module's exercises under `examples/` and run them with `cargo run --example m02_moves`.
5. Run `cargo clippy -- -W clippy::pedantic` on everything. Treat Clippy as a reviewer that knows idiomatic Rust; its suggestions are the fastest way to unlearn C++ habits.

How to use the exercises

- Read the compiler error in full before changing anything. Rust's diagnostics usually name the fix; `rustc --explain E0502` gives the long form.
- When an exercise says *make it compile*, do not reach for `.clone()`, `unsafe`, or `Rc<RefCell<_>>` unless the exercise asks. Those are the three escape hatches C++ programmers overuse.
- Keep tests small: a few `#[test]` functions per exercise, run with `cargo test`.

Roadmap

Module	What it covers	Time
1 · Mental model	What carries over from C++, what changes, expression syntax, integer semantics	30 min
2 · Ownership and moves	Destructive moves, `Copy` vs `Clone`, `Drop`, moving out through `&mut`	40 min
3 · Borrowing and lifetimes	`&T` vs `&mut T`, slices, lifetimes, arenas, reading borrow errors	60 min
4 · Enums and matching	Structs and `impl`, data-carrying enums, patterns, state machines, newtypes	40 min

Module	What it covers	Time
5 · Error handling	`Result` and `?`, error types, `thiserror`/`anyhow`, panics	35 min
6 · Traits and generics	Bounds vs concepts, `dyn` dispatch, associated types, operators, orphan rule	50 min
7 · Smart pointers	`Box`/`Rc`/`Arc`/`Weak`, `Cell`/`RefCell`, `LazyLock`, `Cow`	40 min
8 · Iterators and closures	Iterator adaptors, `collect`, closures and `Fn` traits, collections, UTF-8	45 min
9 · Cargo and modules	Crates and modules, visibility, `Cargo.toml`, workspaces, tests, `build.rs`	35 min
10 · Concurrency	`Send`/`Sync`, scoped threads, `Mutex<T>`, channels, `rayon`, atomics	50 min
11 · Async	Futures, Tokio, `join!`/`select!`, `Send` futures, `Pin`, cancellation	50 min
12 · Unsafe and FFI	Unsafe superpowers, Rust's UB rules, Miri, C FFI, `cxx` bridges	50 min
13 · Macros	`const fn`, const generics, `macro_rules!`, procedural macros	35 min

Module 1 — The mental model shift

Rust keeps C++'s cost model — RAII, value types, zero-cost abstractions — and moves the rules you enforce by code review into the compiler. This module is a reading-only tour of what changes, about 30 minutes; every later module zooms in on one row of it.

1.1 What carries over unchanged

Deterministic destruction works exactly as you expect. A scope guard in C++:

```
{
    std::lock_guard<std::mutex> g(m);
    counter++;
} // unlocked here
```

The Rust equivalent, where the guard *is* the access path to the data:

```
let counter = std::sync::Mutex::new(0u64);
{
    let mut g = counter.lock().unwrap();
    *g += 1;
} // guard dropped, mutex unlocked
```

The same goes for files, sockets, and buffers: their `Drop` impl runs at scope end. There is no garbage collector and no hidden runtime. Generics are monomorphized like templates, structs have no hidden headers, and you can control layout with `#[repr(C)]` or `#[repr(align(64))]`.

1.2 Immutable unless you say otherwise

```
int x = 5;          // mutable
const int y = 6;    // opt in to immutability
```

```
let x = 5;          // immutable
let mut y = 6;      // opt in to mutation
y += x;
// x += 1;          // error[E0384]: cannot assign twice to immutable
 ↪  variable `x`
```

Rust also allows *shadowing*: rebinding a name to a new value, often with a new type. It replaces a lot of `tmp`, `parsed_x`, `x2` naming in C++.

```
let port = "8080";              // &str
let port: u16 = port.parse()?;  // new binding, new type; the old one
 ↪  is gone
```

1.3 Everything is an expression

Blocks, `if`, and `match` all produce values. The last expression in a block, *without* a trailing semicolon, is its value. There is no ternary operator because `if` already is one.

```
int sign = x < 0 ? -1 : (x == 0 ? 0 : 1);
```

```
let sign = if x < 0 { -1 } else if x == 0 { 0 } else { 1 };

let area = {
    let w = rect.right - rect.left;
    let h = rect.bottom - rect.top;
    w * h            // no semicolon: this is the block's value
};

fn square(x: i32) -> i32 {
    x * x           // implicit return; `return` exists for early exits
}
```

A stray semicolon turns the value into `()` (unit, roughly `void`), which is the source of a classic early error: *expected `i32`, found `()`*.

1.4 Assignment moves, not copies

```cpp
std::vector<int> a = {1, 2, 3};
auto b = a;                // deep copy
auto c = std::move(a);     // a is now valid-but-unspecified
a.push_back(4);            // legal, if odd
```

```rust
let a = vec![1, 2, 3];
let b = a.clone();         // deep copy must be spelled out
let c = a;                 // move; `a` is now statically dead
// a.push(4);              // error[E0382]: borrow of moved value: `a`
```

There is no moved-from state to reason about. Module 2 covers this in depth.

1.5 The core rule: shared XOR mutable

At any point, a value can have *either* any number of shared references `&T` *or* exactly one mutable reference `&mut T`. This one rule rules out iterator invalidation, use-after-free through aliases, and data races. The C++ bug it prevents:

```cpp
std::vector<int> v = {1, 2, 3};
for (int& x : v) {
    if (x == 2) v.push_back(4);   // may reallocate; x now dangles
}
```

The Rust version does not compile:

```rust
let mut v = vec![1, 2, 3];
for x in &v {                     // shared borrow of v for the whole loop
    if *x == 2 { v.push(4); }     // needs &mut v while &v is live
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed
 ↪  as immutable
  |
2 | for x in &v {
  |          -- immutable borrow occurs here
3 |     if *x == 2 { v.push(4); }
  |                  ^^^^^^^^^ mutable borrow occurs here
```

The fix is to separate deciding from mutating:

```
let extra = v.iter().filter(|&&x| x == 2).count();
v.extend(std::iter::repeat(4).take(extra));
```

1.6 No null, no exceptions, few implicit conversions

- **No null references.** Absence is `Option<T>`, and you cannot use the inner value without handling `None`.
- **No exceptions.** Fallible functions return `Result<T, E>`; `?` propagates the error. Bugs `panic!`.
- **No implicit numeric conversions and no overloading.** `let y: u64 = x;` with `x: u32` is an error; write `u64::from(x)` or `x.into()`. A function name has exactly one signature.

```
fn find_user(id: u32) -> Option<User> { /* ... */ }
fn load(path: &Path) -> Result<Config, io::Error> { /* ... */ }

let name = match find_user(42) {
    Some(u) => u.name,
    None => "anonymous".to_string(),
};
let cfg = load(Path::new("app.toml"))?;   // early-returns the error
```

1.7 Integers behave predictably

Signed overflow is not UB. In debug builds arithmetic overflow panics; in release it wraps (configurable with `overflow-checks`). When you mean a specific behavior, say so:

```
let a: u8 = 250;
let b = a.wrapping_add(10);      // 4
let c = a.checked_add(10);       // None
let d = a.saturating_add(10);    // 255
let (e, overflowed) = a.overflowing_add(10);  // (4, true)

let idx: usize = 3;              // indices and lengths are usize
let n = idx as u32;              // `as` truncates silently; prefer
 ↪ u32::try_from(idx)?
```

Slice and `Vec` indexing is bounds-checked and panics on out-of-range. Iterators usually let the optimizer remove the checks; `get(i)` returns an `Option` instead of panicking.

1.8 Undefined behavior has an address

In safe Rust, the compiler guarantees no UB. Code that needs raw pointers, FFI, or hand-rolled synchronization goes in `unsafe` blocks, which you can grep for and audit. Module 12 covers what you take responsibility for inside them.

Remember

- RAII, zero-cost generics, and layout control carry over.
- Immutable by default; blocks are expressions.
- Assignment moves; copies are explicit `.clone()`.
- Shared XOR mutable is the rule behind nearly every borrow-checker error.
- `Option` replaces null, `Result` replaces exceptions, and UB is confined to `unsafe`.

Optional exercises

Skip these on a first read; they need a keyboard and `cargo`. Create a scratch crate with `cargo new m01`.

1. **Same bug, two languages.** Write the vector-push-while-iterating bug in C++ and run it under ASan. Port it to Rust and read error E0502 with `rustc --explain E0502`. Fix it two ways: collect then extend, and an index-based `while` loop.

2. **Expression style.** Rewrite this C-style function without `return` or mutable variables:

```rust
fn classify(n: i64) -> &'static str {
    let mut s = "";
    if n < 0 { s = "negative"; }
    else if n == 0 { s = "zero"; }
    else { s = "positive"; }
```

```
    return s;
}
```

3. **Integer semantics.** Write a test that adds `1` to `i32::MAX` and observe it panic under `cargo test` and wrap under `cargo test --release`. Then rewrite with `checked_add` so both behave the same.
4. **Port a function.** Take a 30-line C++ function you know well. Port it without using `.clone()`, `unsafe`, or `Rc`. Note each compiler error and which section of this module it relates to.

Module 2 — Ownership and moves

Every value has exactly one owner, ownership moves on assignment and on pass-by-value, and the owner's scope end runs `Drop`. Think of every non-trivial Rust type as a `std::unique_ptr` whose moved-from husk the compiler deletes for you. About 40 minutes of reading.

2.1 Ownership in one example

```rust
fn main() {
    let s = String::from("hello");   // s owns a heap buffer
    let t = s;                        // ownership moves to t
    println!("{t}");                  // fine
    // println!("{s}");               // error[E0382]: borrow of moved
    ↪ value: `s`
}                                   // t dropped here; buffer freed once
```

The C++ analogue, with the difference called out:

```cpp
std::string s = "hello";
std::string t = std::move(s);   // move constructor runs, s left
↪ valid-but-unspecified
std::cout << s;                 // compiles; prints something (probably
↪ "")
// two destructors run: t's and s's
```

In Rust, a move is always a shallow bitwise copy of the value (for `String`: pointer, length, capacity) and the source is never touched again — its destructor does not run. There are no move constructors to write, and no moved-from state to document.

2.2 Moves into and out of functions

Passing by value moves; returning by value moves back out.

```rust
fn consume(v: Vec<u32>) -> usize {
    v.len()
}                                       // v dropped here

fn make() -> Vec<u32> {
    vec![1, 2, 3]                      // moved out to the caller, no copy
}

let v = make();
let n = consume(v);
// v.len();                             // error: v was moved into consume
```

If the caller still needs the value, the function should borrow it instead — `fn inspect(v: &Vec<u32>)` or, better, `fn inspect(v: &[u32])`. That is Module 3. A useful default for API design:

You want the function to…	Parameter type	C++ analogue
Read it	`&T` (or `&str`, `&[T]`)	`const T&`, `std::string_view`, `std::span`
Modify it in place	`&mut T`	`T&`
Keep it (store it, send it to a thread)	`T`	`T` by value + `std::move` at the call site

2.3 Partial moves and moving out of containers

You can move a field out of a struct you own; the struct becomes partially moved and only its remaining fields are usable.

```rust
struct Request { path: String, body: Vec<u8> }

let r = Request { path: "/".into(), body: vec![] };
let body = r.body;          // partial move
println!("{}", r.path);     // still fine
// let r2 = r;              // error: use of partially moved value
```

You cannot move out of something you only borrow — that would leave a hole the owner doesn't know about. The standard library has three tools for this, each the safe version of a C++ idiom:

```
use std::mem;

fn reset(req: &mut Request) -> Vec<u8> {
    mem::take(&mut req.body)                // moves out, leaves
     ↪ Default (empty Vec)
}

fn swap_path(req: &mut Request, p: String) -> String {
    mem::replace(&mut req.path, p)          // moves out, puts p in
     ↪ its place
}

let mut slot: Option<Request> = Some(Request { path: "/a".into(),
 ↪ body: vec![] });
let taken = slot.take();                    // Option::take leaves
 ↪ None behind
```

The same applies to containers: `v[0]` gives you a place, and `let x = v[0];` only compiles if the element type is `Copy`. To move elements out, use `v.remove(i)`, `v.swap_remove(i)`, `v.pop()`, `v.drain(..)`, or consume the whole vector with `into_iter()`.

2.4 `Copy` and `Clone`

`Clone` is an explicit deep copy — the copy constructor you have to call by name. `Copy` marks types where a bitwise copy is a full, independent copy; for those, assignment copies instead of moving.

```
#[derive(Debug, Clone, Copy, PartialEq)]
struct Point { x: f32, y: f32 }

let a = Point { x: 1.0, y: 2.0 };
let b = a;                  // copy: a still usable
println!("{a:?} {b:?}");

#[derive(Debug, Clone)]
struct Named { name: String, p: Point }

let n = Named { name: "origin".into(), p: a };
```

```
let m = n.clone();          // explicit deep copy of the String
let o = n;                  // move
```

Rules worth memorizing:

- Primitives, `bool`, `char`, shared references `&T`, and tuples/arrays of `Copy` types are `Copy`.
- `String`, `Vec`, `Box`, and `&mut T` are not. A type with a `Drop` impl can never be `Copy`.
- `Copy` is a promise about your public API: removing it later breaks callers. Derive it for small value types (ids, points, handles), not for everything.

2.5 `Drop`: destructors

```
struct Fd(i32);

impl Drop for Fd {
    fn drop(&mut self) {
        // SAFETY: we own this descriptor and close it exactly once.
        unsafe { libc::close(self.0) };
    }
}

fn open_log() -> std::io::Result<Fd> { /* ... */ }

{
    let fd = open_log()?;
    // use fd
} // close() runs here
```

Differences from C++ destructors:

	C++	Rust
Member destruction order	Reverse declaration order	Declaration order, after the struct's own `drop`
Local variable order	Reverse construction order	Reverse declaration order (same)
Moved-from objects	Destructor still runs	No destructor; the value is gone

	C++	Rust
Explicit early destruction	Call `reset()` or scope it	`drop(x)` — just a function that takes ownership
Leaking	Possible, "wrong"	`std::mem::forget(x)` is safe; leaking is not UB
Destructor failures	Exceptions in destructors terminate	`drop` returns `()`; report errors with an explicit `close()` method if they matter

The last row matters for systems code: if closing or flushing can fail meaningfully, expose `fn close(self) -> io::Result<()>` that consumes the value, and keep `Drop` as the best-effort fallback. In the standard library, `BufWriter::into_inner()` surfaces flush errors this way, and `File::sync_all()` lets you observe fsync failures before the silent close in `Drop`.

2.6 Ownership shapes your data structures

Ownership forms a tree: each value has one owner, which may itself be owned. That makes these easy:

```
struct Engine {
    tables: Vec<Table>,              // Engine owns tables
    cache: HashMap<Key, Block>,      // and the cache
}
struct Table { name: String, columns: Vec<Column> }
```

and these awkward: doubly linked lists, graphs with cycles, observers holding back-pointers. In C++ you'd use raw pointers and discipline; in Rust you choose between indices into an arena (Module 3), reference counting with `Rc`/`Weak` (Module 7), or `unsafe` with a safe wrapper (Module 12). Most production Rust picks indices.

2.7 Common mistakes from C++ habits

- **Cloning to silence the compiler.** It works, but it often hides a design question: who should own this? Look for a borrow first.

- **Taking `String` when `&str` would do.** Owned parameters force callers to allocate or give up their value.
- **Expecting `let x = v[i];` to copy a non-`Copy` element.** Borrow it (`&v[i]`) or clone it.
- **Writing a `close()` destructor that can fail silently.** Add an explicit consuming method.

Remember

- One owner; moves are bitwise and destructive; no moved-from state.
- Borrow to read or modify, take by value to keep.
- `Copy` for small value types, `.clone()` for explicit deep copies.
- `Drop` is your destructor; fields drop in declaration order.
- `mem::take`, `mem::replace`, and `Option::take` move out through a `&mut`.

Optional exercises

1. **Two fixes for a move error.** Write `struct Buffer { data: Vec<u8> }` and `fn checksum(b: Buffer) -> u32`. Call it and then use the buffer again. Fix it once by returning ownership `(u32, Buffer)` and once by taking `&Buffer`. Which signature would you ship?

2. **Drop order.** Write a `Noisy(&'static str)` type whose `Drop` prints its name. Put three in a struct and three as locals, then predict and verify the print order. Add `drop(x)` for one local and an `std::mem::forget` for another.

3. **`Copy` boundary.** Derive `Copy` on `Point`, then add a `label: String` field and read the error. Remove `Copy`, keep `Clone`, and fix the call sites.

4. **Moving out through `&mut`.** Implement a small state machine:

```rust
enum Conn { Idle, Busy(Vec<u8>), Closed }
impl Conn {
    fn finish(&mut self) -> Option<Vec<u8>> { todo!() }
}
```

   `finish` should return the buffer and leave the connection `Idle` when it was `Busy`. Use `std::mem::replace`.

5. **Consuming close.** Wrap `std::fs::File` in a `Journal` type with `fn append(&mut self, rec: &[u8])` and `fn close(self) -> io::Result<()>` that flushes and syncs. Verify you cannot call `append` after `close`.

Module 3 — Borrowing and lifetimes

A reference is a C++ reference plus a compile-time proof that it cannot dangle and does not alias a live mutable reference. This is the module that makes or breaks the first month with Rust; read it twice. About 60 minutes.

3.1 Two kinds of reference

```rust
fn total(xs: &[u64]) -> u64 {          // shared borrow: read-only
    xs.iter().sum()
}

fn scale(xs: &mut [u64], k: u64) {      // mutable borrow: exclusive
    for x in xs.iter_mut() { *x *= k; }
}

let mut v = vec![1, 2, 3];
let t = total(&v);                      // borrow, then give back
scale(&mut v, 10);
println!("{t} {v:?}");                  // 6 [10, 20, 30]
```

- `&T` is like `const T&`, but stronger: nobody — not even another alias — can mutate the referent while the borrow is live (with one exception: interior mutability, Module 7).
- `&mut T` has no C++ equivalent. It is a guarantee of exclusivity, roughly `T& __restrict` that the compiler checks. That's why LLVM can optimize Rust code with `noalias` everywhere.
- You create references explicitly with `&` and `&mut` at the call site. Method calls auto-borrow: `v.len()` is `Vec::len(&v)`, `v.push(1)` is `Vec::push(&mut v, 1)`.

3.2 The borrow rules, precisely

1. A value can have any number of `&T` *or* one `&mut T` at a time.
2. A reference must not outlive the value it points to.
3. While a value is borrowed, the owner can't move or mutate it in a way that conflicts.

Borrows last until their *last use*, not until the end of the scope (non-lexical lifetimes). So this compiles:

```
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{first}");      // last use of `first`
v.push(4);                // fine: the shared borrow already ended
```

and this doesn't:

```
let mut v = vec![1, 2, 3];
let first = &v[0];
v.push(4);                // error[E0502]: push may reallocate
println!("{first}");      // ...and first is used after it
```

The second example is exactly the C++ bug where `push_back` invalidates a reference to an element.

3.3 Slices: the native view types

`&str` and `&[T]` are borrowed, length-carrying views — `std::string_view` and `std::span` that can't dangle. Write functions against them, not against `&String` or `&Vec<T>`.

```
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

let line = String::from("GET /index.html HTTP/1.1");
let verb = first_word(&line);        // &String coerces to &str
let path = &line[4..15];             // byte-range slicing
// line.clear();                      // error: verb and path still
 ↪  borrow line
println!("{verb} {path}");
```

The C++ version compiles fine and dangles if `line` is cleared — the classic

`string_view` footgun.

3.4 Lifetimes: naming the relationship

When a function returns a reference, the compiler must know which input it borrows from. Usually it infers this (*elision*). When it can't, you name the relationship with a lifetime parameter.

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

Read `'a` as: *there is some region of code, and the result is valid within the region where both inputs are valid*. You never choose how long `'a` is; callers do, by what they pass in. The compiler then checks every call site:

```
let outer = String::from("long string");
let result;
{
    let inner = String::from("xyz");
    result = longest(&outer, &inner);
}                                   // inner dropped here
// println!("{result}");           // error[E0597]: `inner` does not
 ↪  live long enough
```

Elision rules (why you rarely write lifetimes):

1. Each reference parameter gets its own lifetime.
2. If there is exactly one input lifetime, the output gets it.
3. If there is a `&self` or `&mut self`, the output gets `self`'s lifetime.

```
fn trim(s: &str) -> &str;                 // rule 2: output borrows
 ↪  from s
impl Table { fn name(&self) -> &str; }    // rule 3: output borrows
 ↪  from self
fn pick(a: &str, b: &str) -> &str;        // ambiguous: error, needs 'a
```

3.5 Structs that hold references

A struct holding a reference must declare the lifetime, and can't outlive what it points to. This is how zero-copy parsers are written.

```rust
struct Tokenizer<'a> {
    src: &'a str,
    pos: usize,
}

impl<'a> Iterator for Tokenizer<'a> {
    type Item = &'a str;                        // tokens borrow from
    ↪ src, not from the tokenizer

    fn next(&mut self) -> Option<&'a str> {
        let rest = &self.src[self.pos..];
        let rest_trimmed = rest.trim_start();
        let start = self.pos + (rest.len() - rest_trimmed.len());
        if start >= self.src.len() { return None; }
        let len = self.src[start..]
            .find(char::is_whitespace)
            .unwrap_or(self.src.len() - start);
        self.pos = start + len;
        Some(&self.src[start..start + len])
    }
}

let input = String::from("SELECT a FROM t");
let tokens: Vec<&str> = Tokenizer { src: &input, pos: 0 }.collect();
```

Note the `Item = &'a str`: the tokens borrow from the *input*, so they stay valid after the tokenizer is gone. If the signature were `fn next(&mut self) -> Option<&str>`, elision would tie them to `&mut self` and you could only hold one at a time.

3.6 `'static` and `T: 'static`

- `&'static str` is a reference valid for the whole program — string literals, or deliberately leaked data.
- `T: 'static` as a *bound* means "T contains no borrows shorter than the program". `String`, `Vec<u8>`, and `u64` all satisfy it — they own their data. `&'a str` with a non-static `'a` doesn't.

You'll meet the bound on `std::thread::spawn` and `tokio::spawn`: the spawned task may outlive the caller, so it can't hold borrows into the caller's stack. Move owned data in instead (or use scoped threads, Module 10).

3.7 Fighting the borrow checker: patterns that work

Mutating while iterating. Split decide from apply, or use a purpose-built method.

```
// remove expired sessions
sessions.retain(|s| !s.expired());

// update in place
for s in sessions.iter_mut() { s.touch(); }

// need to push while scanning: collect first
let to_add: Vec<Session> = sessions.iter().filter(|s|
 ↪  s.needs_peer()).map(Session::peer).collect();
sessions.extend(to_add);
```

Two mutable references into one slice. The compiler can't see that indices differ, so split the slice:

```
fn swap_halves(v: &mut [u32]) {
    let mid = v.len() / 2;
    let (a, b) = v.split_at_mut(mid);   // two disjoint &mut slices
    a.swap_with_slice(&mut b[..a.len()]);
}

let [x, y] = v.get_disjoint_mut([2, 5]).unwrap();   // two &mut
 ↪  elements, checked
*x += *y;
```

Method needs `&mut self` but you hold a borrow of a field. Borrow fields directly instead of going through a method; the compiler tracks disjoint fields.

```
struct Server { conns: Vec<Conn>, stats: Stats }

impl Server {
    fn tick(&mut self) {
        for c in &mut self.conns {
            self.stats.record(c);          // OK: disjoint fields
            // self.bump_stats(c);         // error: borrows all of self
        }
    }
}
```

Graphs and back-pointers. Use an arena with indices. Ids are `Copy`, carry no lifetime, and survive serialization.

```rust
#[derive(Clone, Copy, PartialEq, Eq, Hash, Debug)]
struct NodeId(u32);

struct Graph {
    nodes: Vec<Node>,
}
struct Node {
    label: String,
    out: Vec<NodeId>,
    parent: Option<NodeId>,          // back-pointer without a lifetime
}

impl Graph {
    fn add(&mut self, label: &str, parent: Option<NodeId>) -> NodeId {
        let id = NodeId(self.nodes.len() as u32);
        self.nodes.push(Node { label: label.into(), out: vec![], parent
        ↪ });
        if let Some(p) = parent { self.nodes[p.0 as
        ↪ usize].out.push(id); }
        id
    }
    fn node(&self, id: NodeId) -> &Node { &self.nodes[id.0 as usize] }
}
```

This is how `rustc` itself, most ECS game engines, and graph libraries like `petgraph` are built. Deletion needs a free list or generational indices (the `slotmap` crate).

Self-referential structs. A struct that stores a buffer and a slice into that buffer can't be written with plain references, because moving the struct would invalidate the slice. Store offsets (`Range<usize>`) instead.

3.8 Reading borrow-checker errors

The errors you'll see most, and what they usually mean:

Code	Message	Usual cause
E0502	cannot borrow as mutable because it is also borrowed as immutable	Mutating while a reference is live — often iterator invalidation
E0499	cannot borrow as mutable more than once	Two `&mut` paths to the same data; split the borrow
E0505 / E0382	cannot move out because it is borrowed / use of moved value	Ownership handed off while still in use
E0597 / E0515	does not live long enough / returns a reference to local data	Reference outlives its owner; return an owned value
E0106	missing lifetime specifier	Function returns a reference from several inputs; add `'a`

The error output marks each borrow with a label ("first borrow occurs here", "later used here"). Read the labels in order — they tell the story of the conflict.

Remember

- `&T` many, or `&mut T` one; borrows end at last use.
- Write APIs against `&str` and `&[T]`.
- Lifetimes describe which input an output borrows from; elision covers most cases.
- `T: 'static` means "owns its data", not "lives forever".
- For graphs and back-pointers, reach for arenas and indices before `Rc` or `unsafe`.

Optional exercises

1. **Dangling view.** Implement `fn first_word(s: &str) -> &str`. Then call `clear()` on the source `String` while the result is live and read the error. Write the equivalent C++ with `std::string_view` and catch it with ASan.
2. **Zero-copy tokenizer.** Type in the `Tokenizer` from 3.5, then extend it to treat `(`, `)`, and `,` as single-character tokens. Write a test that collects

tokens, drops the tokenizer, and still uses the tokens.

3. **Elision on and off.** Change `type Item = &'a str` to borrow from `&mut self` instead (hint: it can't be done with `Iterator`; try a `fn next_token(&mut self) -> Option<&str>` method). Explain why `collect()` no longer works.
4. **Disjoint mutable borrows.** Write `fn get_two_mut(v: &mut [i32], i: usize, j: usize) -> (&mut i32, &mut i32)` using `split_at_mut`, handling `i > j` and panicking on `i == j`. Compare with `get_disjoint_mut`.
5. **Arena graph.** Extend the `Graph` above with BFS from a node, returning visited labels in order. Then implement the same graph with `Rc<RefCell<Node>>` and `Weak` parents (after Module 7) and compare the code size and ergonomics.
6. **Field-level borrows.** Write a `Server::tick` like the one in 3.7, then refactor the stats update into a `&mut self` method and read the error. Fix it by making the method take `&mut Stats`.

Module 4 — Structs, enums, and pattern matching

Rust enums are `std::variant` with real syntax and exhaustiveness checking, and they end up replacing class hierarchies, tagged unions, sentinel values, and most uses of `std::optional`. About 40 minutes.

4.1 Structs and `impl` blocks

Data and behavior are declared separately. There are no constructors; by convention, an associated function named `new` builds the value.

```cpp
class Rect {
public:
    Rect(double w, double h) : w_(w), h_(h) {}
    double area() const { return w_ * h_; }
    void scale(double k) { w_ *= k; h_ *= k; }
private:
    double w_, h_;
};
```

```rust
pub struct Rect {
    w: f64,                      // private to this module by default
    h: f64,
}

impl Rect {
    pub fn new(w: f64, h: f64) -> Self { Self { w, h } }   // field
    ↪ init shorthand
    pub fn square(s: f64) -> Self { Self::new(s, s) }      // "named
    ↪ constructor"
    pub fn area(&self) -> f64 { self.w * self.h }          // like a
    ↪ const member
    pub fn scale(&mut self, k: f64) { self.w *= k; self.h *= k; }
    pub fn into_tuple(self) -> (f64, f64) { (self.w, self.h) }  //
    ↪ consumes self
```

```
}
```

The receiver tells you everything: `&self` reads, `&mut self` mutates, `self` consumes. There's no `this` pointer and no implicit member access — always `self.w`.

Two more struct shapes:

```
struct Meters(f64);                 // tuple struct: a zero-cost newtype
struct Marker;                       // unit struct: zero-sized, useful
 ↪  as a type tag

let d = Meters(3.5);
let Meters(raw) = d;                 // destructure to get the f64 back
```

Struct update syntax copies remaining fields from another value: `let cfg = Config { port: 9000, ..Config::default() };`.

4.2 Enums carry data

```
enum Shape {
    Circle { r: f64 },
    Rect { w: f64, h: f64 },
    Triangle(f64, f64, f64),
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle { r } => std::f64::consts::PI * r * r,
            Shape::Rect { w, h } => w * h,
            Shape::Triangle(a, b, c) => {
                let s = (a + b + c) / 2.0;
                (s * (s - a) * (s - b) * (s - c)).sqrt()
            }
        }
    }
}
```

The C++ equivalent with `std::variant` needs three structs, an overload set or `if constexpr` chain, and `std::visit`. More importantly, if you add `Shape::Hexagon`, every `match` that doesn't handle it fails to compile — no

forgotten `default:`.

Enum vs trait object. Use an enum when the set of variants is closed and known to you (protocol messages, AST nodes, states). Use a trait (Module 6) when the set is open and others add implementations (plugins, codecs, storage backends). This is the classic expression problem, and Rust makes both sides cheap.

4.3 `Option<T>`: the end of null

```
fn find(users: &[User], id: u32) -> Option<&User> {
    users.iter().find(|u| u.id == id)
}

match find(&users, 7) {
    Some(u) => println!("found {}", u.name),
    None => println!("no such user"),
}

// Or with combinators:
let name = find(&users, 7).map(|u|
 ↪  u.name.as_str()).unwrap_or("anonymous");
```

`Option<&T>`, `Option<Box<T>>`, and `Option<NonZeroU32>` are the same size as the inner type: the compiler uses the null (or zero) bit pattern as `None`. You get null-pointer efficiency with none of the risk.

4.4 Pattern matching in depth

Patterns appear in `match`, `if let`, `while let`, `let`, function parameters, and `for` loops.

```
match packet {
    Packet::Data { stream: 0, .. } => handle_control(),           //
     ↪  literal in a field
    Packet::Data { stream, payload } if payload.is_empty() =>
     ↪  ack(stream), // guard
    Packet::Data { stream, payload } => deliver(stream, payload),
    Packet::Ping(seq @ 0..=99) => pong(seq),                     //
     ↪  bind + range
    Packet::Ping(_) => {}                                         //
     ↪  ignore value
```

```
    Packet::Close(reason) | Packet::Reset(reason) => close(reason), //
     ↪ or-pattern
}
```

Slice patterns parse fixed-format binary data nicely:

```
fn parse_header(buf: &[u8]) -> Option<(u8, u16, &[u8])> {
    match buf {
        [0xCA, 0xFE, ver, hi, lo, rest @ ..] => Some((*ver,
 ↪  u16::from_be_bytes([*hi, *lo]), rest)),
        _ => None,
    }
}
```

Single-pattern shortcuts:

```
if let Some(cfg) = load_config() {
    apply(cfg);
}

let Some(port) = cfg.port else {
    return Err(Error::MissingPort);          // let-else must diverge
};

while let Some(job) = queue.pop() {
    run(job);
}

for (i, (k, v)) in map.iter().enumerate() { /* tuples destructure */ }
```

Match ergonomics. When you match on a reference, bindings automatically become references: matching `&Shape` gives you `r: &f64`, not `f64`. That's why `Shape::area(&self)` above works without `ref` keywords or derefs.

4.5 Fieldless enums and `repr`

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
#[repr(u8)]
enum Opcode {
    Get = 1,
    Put = 2,
```

```
    Delete = 3,
}

impl TryFrom<u8> for Opcode {
    type Error = u8;
    fn try_from(b: u8) -> Result<Self, u8> {
        match b {
            1 => Ok(Opcode::Get),
            2 => Ok(Opcode::Put),
            3 => Ok(Opcode::Delete),
            other => Err(other),
        }
    }
}

let op = Opcode::try_from(buf[0]).map_err(|b| Error::BadOpcode(b))?;
let byte = Opcode::Put as u8;      // enum → int is a cast; int → enum
 ↪  must be checked
```

Unlike `enum class`, transmuting an arbitrary integer into an enum is UB — an enum value must be one of its variants. Always go through a checked conversion.

4.6 Modeling state machines

Enums plus ownership make illegal states unrepresentable. A connection that only has a socket when it's open:

```
enum Conn {
    Disconnected { retries: u32 },
    Connecting { started: Instant },
    Open { sock: TcpStream, peer: SocketAddr },
}

impl Conn {
    fn on_connected(self, sock: TcpStream, peer: SocketAddr) -> Conn {
        match self {
            Conn::Connecting { .. } => Conn::Open { sock, peer },
            other => other,                    // ignore in other states
        }
    }
}
```

There's no `sock` field that's null half the time and no `is_open` flag to keep in sync. Taking `self` by value means the old state is consumed by the transition.

A stronger variant, the *typestate* pattern, uses distinct types per state so invalid calls don't compile at all:

```
struct Request<S> { url: String, headers: Vec<(String, String)>, _s:
↪  PhantomData<S> }
struct Building; struct Ready;

impl Request<Building> {
    fn header(mut self, k: &str, v: &str) -> Self {
    ↪  self.headers.push((k.into(), v.into())); self }
    fn finish(self) -> Request<Ready> { Request { url: self.url,
    ↪  headers: self.headers, _s: PhantomData } }
}
impl Request<Ready> {
    fn send(&self) -> Response { /* ... */ }
}
// Request<Building>::send doesn't exist: a compile error, not a
↪  runtime check.
```

4.7 Newtypes

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct TableId(u32);
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct ColumnId(u32);

fn drop_column(t: TableId, c: ColumnId) { /* ... */ }
// drop_column(col, table);   // error: mismatched types
```

Newtypes compile to the bare `u32`. They're also the standard workaround for implementing foreign traits on foreign types (Module 6).

Remember

- `impl` blocks hold methods; the receiver (`&self`, `&mut self`, `self`) is the contract.
- Enums carry data; `match` must be exhaustive.

- Closed set → enum; open set → trait.
- `Option` replaces null at zero cost for pointer-like types.
- Use enums and typestates to make invalid states unrepresentable; use newtypes for ids.

Optional exercises

1. **Protocol enum.** Model a tiny key-value protocol: `Get { key }`, `Put { key, value }`, `Delete { key }`, `Ok(Option<Vec<u8>>)`, `Err(String)`. Write `encode(&self) -> Vec<u8>` and `decode(&[u8]) -> Option<Msg>` using slice patterns. Add a variant and follow the compiler errors.
2. **Port a hierarchy.** Take a small C++ class hierarchy you know (shapes, AST nodes, log records) and rewrite it as one enum. Note what got simpler and what got harder.
3. **Niches.** Print `std::mem::size_of` for `Option<u32>`, `Option<&u32>`, `Option<Box<u32>>`, `Option<NonZeroU32>`, `Option<Option<bool>>`, and `Shape`. Explain each result.
4. **State machine.** Implement the `Conn` enum from 4.6 with transitions `start()`, `on_connected()`, `on_error()`, and `close()`. Write tests for each transition and one for an invalid transition.
5. **Typestate builder.** Build an `HttpRequest` typestate builder where `send()` is only available after both a method and a URL have been set.

Module 5 — Error handling

Recoverable errors are ordinary return values of type `Result<T, E>`, propagated with `?`; broken invariants `panic!`. If you've used `std::expected` (C++23) monadically, the model will feel familiar — with far less ceremony. About 35 minutes.

5.1 `Result` and `?`

```cpp
// C++23
std::expected<int, std::string> parse_port(std::string_view s) {
    int v{};
    auto [p, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{}) return std::unexpected("not a number");
    if (v <= 0 || v > 65535) return std::unexpected("out of range");
    return v;
}
```

```rust
fn parse_port(s: &str) -> Result<u16, String> {
    let v: u32 = s.trim().parse().map_err(|e| format!("not a number:
    ↪ {e}"))?;
    u16::try_from(v)
        .ok()
        .filter(|&p| p != 0)
        .ok_or_else(|| format!("out of range: {v}"))
}
```

`expr?` means: if `expr` is `Err(e)`, return `Err(From::from(e))` from the current function; otherwise unwrap the `Ok` value. It's a single character where C++ needs an `if` and a return, and it works on `Option` too (returning `None` early).

```rust
fn first_even_square(xs: &[i32]) -> Option<i32> {
    let x = xs.iter().find(|&&x| x % 2 == 0)?;   // None propagates
    Some(x * x)
}
```

`main` can return a `Result` too: `fn main() -> Result<(), Box<dyn std::error::Error>>`. An `Err` from `main` prints its `Debug` form and exits with status 1.

5.2 Designing error types

In a library, define an enum of the things that can go wrong. Implement `Display` (human message), `std::error::Error` (so it composes), and `From` for each underlying error so `?` converts automatically.

By hand:

```
use std::{fmt, io, num::ParseIntError};

#[derive(Debug)]
pub enum ConfigError {
    Io(io::Error),
    Parse { line: usize, source: ParseIntError },
    Missing(&'static str),
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            ConfigError::Io(e) => write!(f, "io error: {e}"),
            ConfigError::Parse { line, source } => write!(f, "line
            ↪  {line}: {source}"),
            ConfigError::Missing(k) => write!(f, "missing key: {k}"),
        }
    }
}

impl std::error::Error for ConfigError {}

impl From<io::Error> for ConfigError {
    fn from(e: io::Error) -> Self { ConfigError::Io(e) }
}
```

With the `thiserror` crate, the same thing:

```
#[derive(Debug, thiserror::Error)]
pub enum ConfigError {
    #[error("io error: {0}")]
    Io(#[from] std::io::Error),
```

```
    #[error("line {line}: {source}")]
    Parse { line: usize, #[source] source: std::num::ParseIntError },
    #[error("missing key: {0}")]
    Missing(&'static str),
}
```

Using it:

```
pub struct Config { pub port: u16, pub workers: usize }

pub fn load(path: &std::path::Path) -> Result<Config, ConfigError> {
    let text = std::fs::read_to_string(path)?;               //
    ↪ io::Error -> ConfigError
    let mut port = None;
    let mut workers = None;
    for (i, line) in text.lines().enumerate() {
        let Some((k, v)) = line.split_once('=') else { continue };
        let parse = |v: &str| v.trim().parse::<usize>()
            .map_err(|source| ConfigError::Parse { line: i + 1, source
            ↪ });
        match k.trim() {
            "port" => port = Some(parse(v)? as u16),
            "workers" => workers = Some(parse(v)?),
            _ => {}
        }
    }
    Ok(Config {
        port: port.ok_or(ConfigError::Missing("port"))?,
        workers: workers.unwrap_or(4),
    })
}
```

5.3 Applications: `anyhow` and context

In binaries, you usually don't match on error variants — you report them. `anyhow::Error` is a boxed, type-erased error with a chain of context messages, a bit like an exception that carries its causes.

```
use anyhow::{Context, Result, bail};

fn run() -> Result<()> {
    let cfg = load("server.conf".as_ref())
```

```
        .context("loading server.conf")?;
    if cfg.workers == 0 {
        bail!("workers must be > 0");
    }
    let sock = std::net::TcpListener::bind(("0.0.0.0", cfg.port))
        .with_context(|| format!("binding port {}", cfg.port))?;
    serve(sock)
}
```

An error printed with `{:#}` shows the whole chain: `loading server.conf: line 3: invalid digit found in string`.

Rule of thumb: libraries return precise error enums (`thiserror`), applications use `anyhow` at the top.

5.4 Panics

`panic!` is for bugs: a violated invariant, an impossible state, out-of-bounds indexing. By default it unwinds the thread, running `Drop` for everything on the stack, and the program exits if it's the main thread. A panic in another thread surfaces as an `Err` from `join()`.

```
# Cargo.toml — abort instead of unwinding (smaller binaries, C++
 ↪  -fno-exceptions feel)
[profile.release]
panic = "abort"
```

What panics in the standard library: indexing out of bounds, `unwrap()`/`expect()` on `None`/`Err`, integer overflow in debug builds, division by zero, `RefCell` double borrow, and explicit `unreachable!()` / `todo!()`.

5.5 `unwrap`, `expect`, and when they're fine

- In tests and quick prototypes: freely.
- When an invariant guarantees success, use `expect` and state the invariant: `let n: u16 = "8080".parse().expect("literal is a valid port")`.
- Locking a `Mutex`: `lock().unwrap()` is idiomatic; the error only occurs if another thread panicked while holding it.
- Everywhere else, propagate with `?` or handle it.

Clippy can enforce this in a crate: `#![deny(clippy::unwrap_used)]`.

5.6 The combinator toolkit

You can always write a `match`, but these read better once familiar:

Method	On	Does
`map(f)`	`Option`, `Result`	Transform the success value
`map_err(f)`	`Result`	Transform the error
`and_then(f)`	both	Chain another fallible step (monadic bind)
`ok_or(e)` / `ok_or_else(f)`	`Option`	Turn `None` into `Err(e)`
`ok()`	`Result`	Drop the error, get an `Option`
`unwrap_or(v)` / `unwrap_or_default()`	both	Fallback value
`filter(p)`	`Option`	Keep `Some` only if predicate holds
`transpose()`	`Option<Result>` ↔ `Result<Option>`	Swap the layers
`collect::<Result<Vec<_>, _>>()`	iterator of `Result`	Stop at the first error

The last one is especially useful:

```
let ports: Result<Vec<u16>, _> = ["80", "443", "x"].iter().map(|s|
↪  s.parse::<u16>()).collect();
assert!(ports.is_err());
```

Remember

- `Result<T, E>` + `?` for recoverable errors; `panic!` for bugs.
- Libraries: an error enum with `Display`, `Error`, and `From` impls (or `thiserror`).
- Applications: `anyhow` with `.context()` for readable error chains.
- `expect("why")` documents an invariant; bare `unwrap()` belongs in tests.

- Collect an iterator of `Results` into `Result<Vec<_>, _>` to stop at the first failure.

Optional exercises

1. **Config loader.** Type in `load` from 5.2 and write tests for a missing file, a bad number, and a missing key. Make the `port` parse reject values above 65535 instead of truncating with `as`.
2. **Hand-rolled vs `thiserror`.** Implement `ConfigError` by hand, then replace it with the `thiserror` version and confirm the tests still pass.
3. **Context chains.** Wrap a call chain three levels deep with `anyhow::Context` and print the error with `{}`, `{:#}`, and `{:?}`. Note what each shows.
4. **Combinators.** Rewrite this using combinators, then decide which version you'd keep:

```
fn user_email(db: &Db, id: u32) -> Option<String> {
    match db.user(id) {
        Some(u) => match u.email {
            Some(e) => if e.contains('@') {
             ↪  Some(e.to_lowercase()) } else { None },
            None => None,
        },
        None => None,
    }
}
```

5. **Panic boundaries.** Spawn a thread that panics, `join()` it, and print the panic message from the `Err`. Then set `panic = "abort"` and observe the difference.

Module 6 — Traits and generics

A trait is a C++20 concept that's checked at the definition site, can provide default methods, and can double as a virtual interface. Generic functions monomorphize like templates, but the body type-checks once against its bounds — no duck typing and no instantiation-time error novels. About 50 minutes.

6.1 Defining and implementing a trait

```rust
pub trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> String { "shape".to_string() }   // default method
}

pub struct Circle { pub r: f64 }
pub struct Square { pub s: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.r * self.r }
    fn name(&self) -> String { format!("circle(r={})", self.r) }
}

impl Shape for Square {
    fn area(&self) -> f64 { self.s * self.s }          // uses default
    ↪ name()
}
```

Implementations live outside the type definition, so you can implement your trait for types you didn't write — including `u64`, `Vec<T>`, or a type from another crate.

6.2 Static dispatch: generics with bounds

```cpp
template <typename T>
    requires requires(const T& t) { { t.area() } ->
     ↪  std::convertible_to<double>; }
double total_area(const std::vector<T>& xs) {
    double s = 0;
    for (auto& x : xs) s += x.area();
    return s;
}
```

```rust
fn total_area<T: Shape>(xs: &[T]) -> f64 {
    xs.iter().map(|x| x.area()).sum()
}

// equivalent spellings
fn total_area2<T>(xs: &[T]) -> f64 where T: Shape { /* ... */ }
fn describe(x: &impl Shape) -> String { x.name() }      // impl Trait
 ↪  in argument position
```

The key difference from templates: inside `total_area`, you can *only* use what `Shape` (and other listed bounds) provide. Calling `x.perimeter()` is an error at definition time, even if every type you'll ever pass has one. Errors come out short and point at the missing bound.

Multiple bounds combine with `+`:

```rust
use std::fmt::Debug;
use std::hash::Hash;

fn dedup_log<K>(keys: Vec<K>) -> Vec<K>
where
    K: Eq + Hash + Clone + Debug,
{
    let mut seen = std::collections::HashSet::new();
    keys.into_iter()
        .filter(|k| { let new = seen.insert(k.clone()); if !new {
         ↪  eprintln!("dup {k:?}"); } new })
        .collect()
}
```

6.3 Dynamic dispatch: trait objects

```
let shapes: Vec<Box<dyn Shape>> = vec![
    Box::new(Circle { r: 1.0 }),
    Box::new(Square { s: 2.0 }),
];
let total: f64 = shapes.iter().map(|s| s.area()).sum();

fn print_all(shapes: &[&dyn Shape]) {
    for s in shapes { println!("{} {:.2}", s.name(), s.area()); }
}
```

`&dyn Shape` is a *fat pointer*: data pointer plus vtable pointer. Unlike C++, the object itself carries no vptr — a `Circle` is just an `f64`. The vtable is attached when you create the trait object, so the same value can be viewed through several traits.

	Generics (`T: Shape`)	Trait objects (`dyn Shape`)
Dispatch	Static, inlinable	Indirect call through vtable
Code size	One copy per type	One copy
Heterogeneous collections	No	Yes
C++ analogue	Templates + concepts	Virtual functions

Dyn compatibility (object safety). A trait can be used as `dyn Trait` only if its methods don't return `Self`, don't take generic type parameters, and take `self` by reference. `Clone` isn't dyn-compatible, for example. Mark offending methods `where Self: Sized` to exclude them from the vtable.

6.4 Associated types and generic traits

An associated type is an output chosen by the implementation — like a nested `using value_type = ...` that the trait requires.

```
pub trait Codec {
    type Item;
    type Error: std::error::Error;
    fn encode(&self, item: &Self::Item, out: &mut Vec<u8>);
    fn decode(&self, buf: &[u8]) -> Result<(Self::Item, usize),
    ↪  Self::Error>;
}
```

```
struct VarIntCodec;
impl Codec for VarIntCodec {
    type Item = u64;
    type Error = std::io::Error;
    fn encode(&self, item: &u64, out: &mut Vec<u8>) { /* LEB128 */ }
    fn decode(&self, buf: &[u8]) -> Result<(u64, usize), Self::Error> {
     ↪  /* ... */ }
}

fn roundtrip<C: Codec>(c: &C, item: &C::Item) -> Result<C::Item,
 ↪  C::Error> {
    let mut buf = Vec::new();
    c.encode(item, &mut buf);
    c.decode(&buf).map(|(v, _)| v)
}
```

Use an associated type when each implementing type has one natural choice (`Iterator::Item`). Use a generic parameter when a type can implement the trait several ways (`From<T>`: `String` implements `From<&str>`, `From<char>`, and more).

6.5 Operators and standard traits

Operator overloading is trait implementation:

```
use std::ops::{Add, Mul, Index};

#[derive(Debug, Clone, Copy, PartialEq, Default)]
struct Vec3 { x: f32, y: f32, z: f32 }

impl Add for Vec3 {
    type Output = Vec3;
    fn add(self, o: Vec3) -> Vec3 { Vec3 { x: self.x + o.x, y: self.y +
     ↪  o.y, z: self.z + o.z } }
}
impl Mul<f32> for Vec3 {
    type Output = Vec3;
    fn mul(self, k: f32) -> Vec3 { Vec3 { x: self.x * k, y: self.y * k,
     ↪  z: self.z * k } }
}
impl Index<usize> for Vec3 {
```

```
    type Output = f32;
    fn index(&self, i: usize) -> &f32 { match i { 0 => &self.x, 1 =>
     ↪  &self.y, 2 => &self.z, _ => panic!("index {i}") } }
}

let v = Vec3 { x: 1.0, y: 2.0, z: 3.0 } * 2.0 + Vec3::default();
assert_eq!(v[1], 4.0);
```

The standard traits you'll use daily, and what derives them:

Trait	Purpose	C++ analogue	Derivable
`Debug`	`{:?}` formatting	`operator<<` for debugging	Yes
`Display`	`{}` formatting	`operator<<` / `std::formatter`	No
`Clone` / `Copy`	Explicit / implicit copy	Copy constructor	Yes
`Default`	Zero/empty value	Default constructor	Yes
`PartialEq` / `Eq`	`==`	`operator==`	Yes
`PartialOrd` / `Ord`	`<`, sorting	`operator<=>`	Yes
`Hash`	Hash map keys	`std::hash` specialization	Yes
`From` / `Into`	Infallible conversion	Converting constructor	No
`TryFrom` / `TryInto`	Fallible conversion	Factory returning `expected`	No
`AsRef` / `Borrow`	Cheap reference conversion	Implicit `string_view` conversion	No
`Deref`	Smart-pointer `*` and auto-deref	`operator*`, `operator->`	No
`Drop`	Destructor	Destructor	No

Implement `From<A> for B` and you get `Into<B> for A` for free. Functions often accept `impl Into<String>` so callers can pass `&str` or `String`.

6.6 Blanket and conditional implementations

You can implement a trait for every type that satisfies a bound — something C++ approximates with partial specialization and SFINAE.

```rust
trait Describe { fn describe(&self) -> String; }

impl<T: std::fmt::Debug> Describe for T {
    fn describe(&self) -> String { format!("<{self:?}>") }
}

// Conditional impl: Pair<T> is comparable only when T is
struct Pair<T>(T, T);
impl<T: PartialOrd + Copy> Pair<T> {
    fn max(&self) -> T { if self.0 > self.1 { self.0 } else { self.1 } }
}
```

The standard library does this constantly: `impl<T: Display> ToString for T` is why every displayable type has `.to_string()`.

6.7 Coherence and the orphan rule

You may implement trait `Tr` for type `Ty` only if your crate defines `Tr` or `Ty`. This guarantees there's never more than one impl in the whole program, so adding a dependency can't change which code runs.

```rust
// impl std::fmt::Display for Vec<u8> { ... }   // error[E0117]: only
 ↪  traits defined in the current crate...

struct Hex<'a>(&'a [u8]);                       // newtype you own
impl std::fmt::Display for Hex<'_> {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        for b in self.0 { write!(f, "{b:02x}")?; }
        Ok(())
    }
}
println!("{}", Hex(&[0xde, 0xad]));             // dead
```

There's no template specialization on stable Rust. When you'd specialize, use a separate trait, an enum, or a marker trait.

6.8 `impl Trait` in return position

```rust
fn evens(limit: u32) -> impl Iterator<Item = u32> {
    (0..limit).filter(|n| n % 2 == 0)
```

```
}
```

The caller sees an opaque type that implements `Iterator`; the compiler knows the concrete type and can inline everything. It's like returning `auto` constrained by a concept, and it's how you return closures and iterator chains without boxing.

Remember

- Traits are checked concepts plus optional vtables.
- `T: Trait` → static dispatch; `dyn Trait` → dynamic dispatch with a fat pointer.
- Associated types for one natural choice, generic parameters for many.
- Operators and conversions are trait impls; derive the standard ones liberally.
- The orphan rule means newtypes when you need a foreign trait on a foreign type.

Optional exercises

1. **Storage backends.** Define `trait Storage { fn get(&self, k: &[u8]) -> Option<Vec<u8>>; fn put(&mut self, k: &[u8], v: &[u8]); fn delete(&mut self, k: &[u8]); }`. Implement it for an in-memory `HashMap` and an append-only file. Write one generic consumer and one using `Box<dyn Storage>`.
2. **Measure dispatch.** Benchmark the generic and `dyn` consumers with `criterion`, and compare binary sizes with `cargo bloat --release`.
3. **Iterator impl.** Implement `Iterator` for a `Fibonacci` struct, then use `.take_while(|&x| x < 1_000_000).filter(|x| x % 2 == 0).sum::<u64>()`. Paste it into Compiler Explorer with `-O` and look at the loop.
4. **Orphan rule.** Try `impl Display for Vec<u8>`, read the error, and fix it with the `Hex` newtype. Add a `{:#}` alternate form that inserts spaces between bytes.
5. **VarInt codec.** Finish `VarIntCodec` (LEB128) from 6.4 and write a property test with `proptest` that `decode(encode(x)) == x` for all `u64`.

Module 7 — Smart pointers and interior mutability

Rust's smart pointers map almost one-to-one onto C++'s. The twist: shared ownership only gives shared (read-only) access, so mutating through a shared pointer requires a cell type that enforces the aliasing rule at runtime instead of compile time. About 40 minutes.

7.1 The map

C++	Rust	Notes
`std::unique_ptr<T>`	`Box<T>`	Never null; `Option<Box<T>>` is the nullable form, same size
`std::shared_ptr<T>`, one thread	`Rc<T>`	Non-atomic refcount; the compiler stops it crossing threads
`std::shared_ptr<T>`, many threads	`Arc<T>`	Atomic refcount
`std::weak_ptr<T>`	`rc::Weak<T>`, `sync::Weak<T>`	`upgrade()` returns `Option`
`mutable` member	`Cell<T>`, `RefCell<T>`	Single-threaded interior mutability
`std::mutex` + the data it guards	`Mutex<T>`	Data lives inside the lock
`std::shared_mutex`	`RwLock<T>`	
`std::atomic<T>`	`AtomicU64`, `AtomicBool`, `AtomicPtr<T>` …	Same memory orderings
Function-local `static`	`OnceLock<T>`, `LazyLock<T>`	Thread-safe one-time init
Copy-on-write string	`Cow<'a, str>`	Borrowed or owned

7.2 Box<T>: one owner on the heap

Use `Box` when you need indirection: recursive types, large values you don't want on the stack, or trait objects.

```
enum Expr {
    Num(f64),
    Add(Box<Expr>, Box<Expr>),        // recursive: needs a known size
    Mul(Box<Expr>, Box<Expr>),
}

fn eval(e: &Expr) -> f64 {
    match e {
        Expr::Num(n) => *n,
        Expr::Add(a, b) => eval(a) + eval(b),   // &Box<Expr> derefs to
         ↪  &Expr
        Expr::Mul(a, b) => eval(a) * eval(b),
    }
}

let e = Expr::Add(Box::new(Expr::Num(2.0)),
 ↪  Box::new(Expr::Mul(Box::new(Expr::Num(3.0)),
 ↪  Box::new(Expr::Num(4.0)))));
assert_eq!(eval(&e), 14.0);
```

`Box<[T]>` and `Box<str>` are fixed-size heap buffers without `Vec`'s capacity field — handy for immutable data.

7.3 Rc<T> and Arc<T>: shared ownership

```
use std::rc::Rc;

let schema = Rc::new(Schema::load("users.avsc"));
let reader_a = Reader { schema: Rc::clone(&schema) };   // refcount 2
let reader_b = Reader { schema: Rc::clone(&schema) };   // refcount 3
println!("{}", Rc::strong_count(&schema));             // 3
```

`Rc::clone` bumps a counter; it doesn't copy the `Schema`. Writing `Rc::clone(&x)` rather than `x.clone()` is a convention that makes the cheap clone obvious.

You get shared access only: `Rc<T>` derefs to `&T`, never `&mut T`. That's the

shared-XOR-mutable rule applied to ownership. If the value needs to change, see 7.5. `Arc<T>` is the same API with an atomic count, and the only one of the two that `thread::spawn` will accept.

`Arc::make_mut` and `Rc::make_mut` give copy-on-write semantics: mutate in place if you're the only owner, otherwise clone first.

7.4 `Weak<T>`: breaking cycles

Reference cycles leak with `Rc` just as they do with `shared_ptr`. Point upward with `Weak`:

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct TreeNode {
    value: i32,
    parent: RefCell<Weak<TreeNode>>,
    children: RefCell<Vec<Rc<TreeNode>>>,
}

let root = Rc::new(TreeNode { value: 1, parent:
 ↪  RefCell::new(Weak::new()), children: RefCell::new(vec![]) });
let leaf = Rc::new(TreeNode { value: 2, parent:
 ↪  RefCell::new(Weak::new()), children: RefCell::new(vec![]) });

*leaf.parent.borrow_mut() = Rc::downgrade(&root);
root.children.borrow_mut().push(Rc::clone(&leaf));

if let Some(p) = leaf.parent.borrow().upgrade() {
    println!("leaf's parent = {}", p.value);
}
```

This works, but notice the noise. Compare with the arena-and-indices version in Module 3; for most trees and graphs, the arena wins.

7.5 Interior mutability: `Cell` and `RefCell`

Sometimes a logically-const operation needs to mutate something internally — a cache, a counter, a lazily built index. C++ uses `mutable`. Rust uses a cell type, which lets you mutate through `&self`.

`Cell<T>` works by moving values in and out; there's never a reference to the inside, so it's always safe. Best for small `Copy` types.

```
use std::cell::Cell;

struct Stats { hits: Cell<u64> }
impl Stats {
    fn record(&self) { self.hits.set(self.hits.get() + 1); }   //
     ↪ &self, still mutates
}
```

`RefCell<T>` hands out references and tracks borrows at runtime. Violating shared-XOR-mutable panics instead of failing to compile.

```
use std::cell::RefCell;
use std::collections::HashMap;

struct Resolver {
    cache: RefCell<HashMap<String, IpAddr>>,
}

impl Resolver {
    fn resolve(&self, host: &str) -> IpAddr {
        if let Some(ip) = self.cache.borrow().get(host) {          //
         ↪ shared borrow, ends here
            return *ip;
        }
        let ip = dns_lookup(host);
        self.cache.borrow_mut().insert(host.to_string(), ip);      //
         ↪ exclusive borrow
        ip
    }
}
```

The failure mode:

```
let c = RefCell::new(vec![1]);
let r = c.borrow();
c.borrow_mut().push(2);    // panics: already borrowed: BorrowMutError
```

`Cell` and `RefCell` are not `Sync`, so they can't be shared across threads. Their thread-safe counterparts are atomics, `Mutex`, and `RwLock` (Module 10).

7.6 Common combinations

Need	Single thread	Multiple threads
Shared, read-only	`Rc<T>`	`Arc<T>`
Shared, mutable	`Rc<RefCell<T>>`	`Arc<Mutex<T>>` or `Arc<RwLock<T>>`
Shared counter or flag	`Rc<Cell<u64>>`	`Arc<AtomicU64>`
Lazily initialized global	`thread_local!` + `OnceCell`	`static X: LazyLock<T>`

Treat `Rc<RefCell<T>>` as a design smell to examine, not a ban. It's right for GUI-style object graphs and some interpreters; for data structures you control, ownership plus indices is usually clearer and faster.

7.7 Globals done right

```
use std::sync::LazyLock;
use std::collections::HashMap;

static MIME: LazyLock<HashMap<&'static str, &'static str>> =
 ↪  LazyLock::new(|| {
    HashMap::from([("html", "text/html"), ("json",
     ↪  "application/json")])
});

fn mime_for(ext: &str) -> &'static str {
    MIME.get(ext).copied().unwrap_or("application/octet-stream")
}
```

No static-initialization-order fiasco: `LazyLock` initializes on first access, exactly once, thread-safely. `static mut` exists but requires `unsafe` for every access; avoid it.

7.8 `Cow`: allocate only when needed

```
use std::borrow::Cow;
```

```rust
fn escape_csv(field: &str) -> Cow<'_, str> {
    if field.contains([',', '"', '\n']) {
        Cow::Owned(format!("\"{}\"", field.replace('"', "\"\"")))
    } else {
        Cow::Borrowed(field)            // common case: no allocation
    }
}
```

7.9 `Deref` and auto-deref

Smart pointers implement `Deref`, which powers `*p` and lets method calls see through them: `Box<String>` → `String` → `str`, so `b.len()` just works. Function arguments get *deref coercion*: `&Box<String>` passes where `&str` is expected.

Don't implement `Deref` to emulate inheritance (a `Derived` that derefs to `Base`). It's a known anti-pattern; use composition and delegate the methods you need, or a trait.

Remember

- `Box` = `unique_ptr`, `Rc`/`Arc` = `shared_ptr`, `Weak` = `weak_ptr`.
- Shared ownership means shared access; mutation needs `Cell`, `RefCell`, a lock, or atomics.
- `RefCell` moves borrow checking to runtime and panics on conflict.
- `LazyLock` replaces function-local statics and init-order hazards.
- Prefer ownership plus indices over `Rc<RefCell<T>>` for your own data structures.

Optional exercises

1. **Expression tree.** Extend the `Expr` enum with `Neg`, `Var(String)`, and `Let { name, value, body }`. Write `eval(&Expr, &HashMap<String, f64>)`.
2. **Doubly linked list.** Build one with `Rc<RefCell<Node>>` and `Weak` back-pointers: `push_front`, `push_back`, `pop_front`, and a forward iterator. Then read *Learning Rust With Entirely Too Many Linked Lists* (rust-unofficial.github.io/too-many-lists).

3. **Trigger the panic.** Write code that panics with `BorrowMutError`, then restructure it so the borrows don't overlap. Use `try_borrow_mut` to handle the conflict gracefully.
4. **Memoizing resolver.** Implement `Resolver` from 7.5 with a fake lookup and a hit counter in a `Cell`. Write a test that the second lookup hits the cache.
5. **Copy-on-write config.** Store a config in `Arc<Config>`, hand clones to several workers, and implement an update that uses `Arc::make_mut`. Verify that workers holding the old `Arc` still see the old values.

Module 8 — Iterators, closures, and collections

Idiomatic Rust leans on lazy iterator chains much more than C++ leans on ranges, and they compile to the same tight loops — often with bounds checks removed, which index loops don't get. About 45 minutes.

8.1 The `Iterator` trait

The whole protocol is one required method:

```
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // ~75 provided adaptor methods: map, filter, zip, fold, ...
}
```

Compare with a C++ iterator pair: no `begin`/`end`, no sentinel comparison, no invalidation — the iterator borrows the collection, so the collection can't change underneath it.

```
struct Countdown(u32);

impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.0 == 0 { return None; }
        self.0 -= 1;
        Some(self.0 + 1)
    }
}

let v: Vec<u32> = Countdown(3).collect();   // [3, 2, 1]
```

8.2 Three ways to iterate a collection

Call	Yields	Collection afterward	`for` sugar
`v.iter()`	`&T`	Unchanged	`for x in &v`
`v.iter_mut()`	`&mut T`	Modified in place	`for x in &mut v`
`v.into_iter()`	`T`	Consumed (moved)	`for x in v`

```rust
let mut names = vec![String::from("ada"), String::from("linus")];

for n in &names { println!("{n}"); }              // borrow
for n in &mut names { n.make_ascii_uppercase(); } // mutate in place
for n in names { consume(n); }                    // move each String out
// names is gone now
```

Forgetting the `&` in `for x in v` and then using `v` afterward is one of the most common early errors.

8.3 Adaptor chains

Adaptors are lazy; nothing runs until a consumer (`collect`, `sum`, `for`, `count`, `fold`…) pulls values.

```cpp
// C++20 ranges
auto total = records
    | std::views::filter([](const Record& r) { return r.status ==
    ↪  Status::Ok; })
    | std::views::transform(&Record::bytes);
uint64_t sum = std::accumulate(total.begin(), total.end(),
 ↪  uint64_t{0});
```

```rust
let sum: u64 = records
    .iter()
    .filter(|r| r.status == Status::Ok)
    .map(|r| r.bytes)
    .sum();
```

A tour of adaptors worth knowing:

```rust
let xs = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
xs.iter().enumerate();                          // (0, &3), (1, &1), ...
xs.iter().zip(ys.iter());                       // pairs
xs.iter().rev();                                // backward (double-ended
↪ iterators)
xs.iter().skip(2).take(3);                      // 4, 1, 5
xs.iter().step_by(2);                           // 3, 4, 5, 2
xs.iter().take_while(|&&x| x < 5);              // 3, 1, 4, 1
xs.windows(3);                                  // overlapping slices
↪ [3,1,4], [1,4,1], ...
xs.chunks(3);                                   // [3,1,4], [1,5,9], [2,6]
xs.iter().flat_map(|&x| 0..x);                  // flatten nested iterators
xs.iter().filter_map(|&x| (x % 2 == 0).then_some(x * 10));  // filter +
↪ map in one
xs.iter().scan(0, |acc, &x| { *acc += x; Some(*acc) });     // running
↪ sum
xs.iter().peekable();                           // look ahead with .peek()
xs.iter().chain(ys.iter());                     // concatenate
```

Consumers:

```
xs.iter().sum::<i32>();
xs.iter().max();                                // Option<&i32>
xs.iter().position(|&x| x == 5);                // Option<usize>
xs.iter().any(|&x| x > 8);
xs.iter().all(|&x| x > 0);
xs.iter().count();
xs.iter().fold(0, |acc, &x| acc * 10 + x);
xs.iter().min_by_key(|&&x| (x as i32 - 4).abs());
let (even, odd): (Vec<i32>, Vec<i32>) = xs.iter().partition(|&&x| x %
↪ 2 == 0);
```

8.4 `collect` and `FromIterator`

`collect` builds whatever container you ask for, guided by type annotations:

```
let v: Vec<_> = (1..=5).collect();
let s: String = ['h', 'i'].iter().collect();
let set: HashSet<_> = words.iter().copied().collect();
let map: HashMap<_, _> = names.iter().cloned().zip(0..).collect();
let joined = parts.join(",");                       // for slices of
↪ strings
```

```
let r: Result<Vec<u16>, _> = inputs.iter().map(|s|
↪  s.parse::<u16>()).collect();
```

The turbofish `::<>` names a type inline when there's no annotation to infer from: `.collect::<Vec<_>>()`.

8.5 Closures

Closures are lambdas with inferred captures. The compiler captures each variable in the least powerful way that works: by shared reference, then by mutable reference, then by value.

```
let threshold = 100;
let big = |x: &u64| *x > threshold;            // captures &threshold

let mut count = 0;
let mut tally = |x: u64| { count += x; };      // captures &mut count
tally(5);

let name = String::from("worker-1");
let h = std::thread::spawn(move || {           // `move`: capture by
↪  value
    println!("{name} running");
});
```

`move` is like `[=]`, but moves instead of copying. It's required when the closure outlives the current stack frame (threads, async tasks, returned closures).

Each closure has a unique anonymous type, which implements one or more of three traits:

Trait	Can be called	Captured state	Example
`Fn`	Many times, concurrently	Read only	Predicate, comparator
`FnMut`	Many times, not concurrently	Mutated	Accumulator, RNG
`FnOnce`	Once	Consumed (moved out)	Thread body, callback that sends a value

Taking closures as parameters:

```
fn retry<T, E>(mut attempts: u32, mut op: impl FnMut() -> Result<T, E>)
 ↪  -> Result<T, E> {
    loop {
        match op() {
            Ok(v) => return Ok(v),
            Err(e) if attempts == 1 => return Err(e),
            Err(_) => attempts -= 1,
        }
    }
}

struct EventBus { handlers: Vec<Box<dyn Fn(&Event) + Send>> }   // like
 ↪  std::vector<std::function>

fn make_adder(n: i32) -> impl Fn(i32) -> i32 { move |x| x + n }
```

`impl Fn` monomorphizes (zero-cost); `Box<dyn Fn>` is the `std::function` equivalent when you need to store different closures together.

8.6 Collections

C++	Rust	Notes
`std::vector`	`Vec<T>`	
`std::array<T, N>`	`[T; N]`	Arrays are values; `Copy` if `T: Copy`
`std::deque`	`VecDeque<T>`	Ring buffer
`std::list`	`LinkedList<T>`	Rarely the right choice, same as in C++
`std::unordered_map` / `set`	`HashMap` / `HashSet`	SipHash by default for DoS resistance; swap in `ahash` or `rustc-hash` for speed
`std::map` / `set`	`BTreeMap` / `BTreeSet`	B-tree: better cache behavior than red-black
`std::priority_queue`	`BinaryHeap<T>`	Max-heap; wrap items in `Reverse` for min
`std::string`	`String`	Always UTF-8

The `entry` API replaces find-then-insert with one lookup:

```rust
let mut counts: HashMap<&str, usize> = HashMap::new();
for w in text.split_whitespace() {
    *counts.entry(w).or_insert(0) += 1;
}

let mut groups: HashMap<u32, Vec<Row>> = HashMap::new();
for r in rows { groups.entry(r.shard).or_default().push(r); }
```

8.7 Strings are UTF-8

`String` and `&str` are guaranteed valid UTF-8. You index by byte range, not by character, because character boundaries aren't O(1) to find.

```rust
let s = "naïve";
assert_eq!(s.len(), 6);                    // bytes, ï is 2 bytes
assert_eq!(s.chars().count(), 5);          // Unicode scalar values
let first = &s[0..1];                      // "n"
// let bad = &s[0..3];                   // panics: byte 3 is inside 'ï'
for (i, c) in s.char_indices() { /* byte offset + char */ }
```

For binary or not-necessarily-UTF-8 data, use `Vec<u8>` / `&[u8]`; for OS strings and paths, `OsString` and `PathBuf`.

8.8 Performance notes

- Iterator chains usually compile to the same code as a hand-written loop, and often vectorize. Check with Compiler Explorer if in doubt.
- `xs[i]` in a loop bounds-checks every access unless the optimizer can prove it's in range; iterating directly avoids the question.
- `Vec::with_capacity(n)` and `reserve` work as in C++; `collect` on an iterator with a known size hint preallocates automatically.
- `extend` and `retain` are the efficient bulk operations; `drain(..)` moves a range out.

Remember

- `iter`, `iter_mut`, `into_iter` → `&T`, `&mut T`, `T`.
- Adaptors are lazy; a consumer drives them.

- `collect` into whatever type you annotate, including `Result<Vec<_>, _>`.
- Closures infer captures; `move` forces by-value; `Fn`/`FnMut`/`FnOnce` describe how they're called.
- Use the `entry` API for maps; strings index by byte range.

Optional exercises

1. **Log report.** Parse lines like `2026-10-01T12:00:00Z GET /a 200 1532` into a struct, then compute the top 10 paths by total bytes and the error rate per hour — no `for` loops.
2. **Lifetimes through iterators.** Write `fn words_with_len<'a>(text: &'a str, n: usize) -> impl Iterator<Item = &'a str> + 'a`.
3. **Three callback styles.** Write a function that accepts a callback as `impl Fn(u64) -> u64`, `&dyn Fn(u64) -> u64`, and `Box<dyn Fn(u64) -> u64>`. Benchmark all three with `criterion` in a hot loop.
4. **Custom iterator.** Implement an iterator over the set bits of a `u64` (yield bit indices) using `trailing_zeros`. Make it `DoubleEndedIterator` too.
5. **UTF-8.** Write `fn truncate_chars(s: &str, max: usize) -> &str` that never splits a character, using `char_indices`.

Module 9 — Crates, modules, and Cargo

Cargo is build system, package manager, test runner, benchmark harness, and doc generator in one, and every Rust project uses it. There are no headers: the crate is the unit of compilation, and modules inside it control naming and visibility. About 35 minutes.

9.1 Vocabulary

Rust	Closest C++ idea
Crate — one library or binary, compiled as a unit	A library target in CMake (not a translation unit)
Module (`mod`) — namespace + privacy boundary	`namespace` + the public/private split of headers
Package — a `Cargo.toml` with one or more crates	A CMake project
Workspace — packages sharing a `Cargo.lock` and `target/`	A monorepo top-level `CMakeLists.txt`
crates.io	Conan/vcpkg central registry
Feature — named, additive compile-time option	`option()` + `#ifdef`

9.2 Modules and files

A crate starts at `src/lib.rs` (library) or `src/main.rs` (binary). Modules are declared with `mod`, which tells the compiler to load a file:

```
src/
├── lib.rs          // mod net; mod storage;
├── net.rs          // or net/mod.rs
├── net/
```

```
│   └── frame.rs    // declared in net.rs as: mod frame;
└── storage.rs
```

```rust
// src/lib.rs
pub mod net;                  // public module
mod storage;                  // private to the crate

pub use storage::Engine;      // re-export: users see `mycrate::Engine`

// src/net.rs
mod frame;                    // private submodule
pub use frame::Frame;

pub fn connect(addr: &str) -> std::io::Result<Conn> { /* ... */ }
pub(crate) fn internal_helper() {}   // visible in this crate only
```

Paths: `crate::net::Frame` (absolute from the crate root), `super::` (parent module), `self::` (current). `use` brings names into scope like a C++ `using` declaration.

Visibility is per item and defaults to private to the module:

Modifier	Visible to
(none)	The current module and its children
`pub(super)`	The parent module
`pub(crate)`	The whole crate — the "internal header" equivalent
`pub`	Everyone who can reach the module

Struct fields have their own visibility. A `pub struct` with private fields can only be constructed inside its module, so a `new` function becomes the enforced constructor.

9.3 `Cargo.toml`

```toml
[package]
name = "kvstore"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"
```

```toml
[dependencies]
bytes = "1"                                  # semver: >=1.0.0, <2.0.0
serde = { version = "1", features = ["derive"] }
zstd = { version = "0.13", optional = true }  # only with the feature
↪ below

[dev-dependencies]                           # tests and benches only
criterion = "0.5"
proptest = "1"

[features]
default = []
compression = ["dep:zstd"]

[profile.release]
lto = "thin"
codegen-units = 1
debug = "line-tables-only"                   # symbols for perf and
↪ flamegraphs
```

`Cargo.lock` pins exact versions; commit it for binaries and workspaces. The `edition` field selects language-level defaults (like `-std=c++20`), and crates on different editions link together fine.

9.4 Workspaces

```toml
# Cargo.toml at the repo root
[workspace]
members = ["kv-core", "kv-server", "kv-cli"]
resolver = "3"

[workspace.dependencies]
tokio = { version = "1", features = ["full"] }
anyhow = "1"

# kv-server/Cargo.toml
[dependencies]
kv-core = { path = "../kv-core" }
tokio.workspace = true
anyhow.workspace = true
```

One `target/` directory, one lockfile, and `cargo test --workspace` runs

everything. Split crates for compile-time parallelism and clean API boundaries, much as you'd split CMake targets.

9.5 Commands you'll use daily

Command	Purpose
`cargo check`	Type-check without codegen — the fast inner loop
`cargo build [--release]`	Build into `target/debug` or `target/release`
`cargo run -- args`	Build and run a binary
`cargo test [name]`	Unit, integration, and doc tests
`cargo bench`	Benchmarks (with `criterion`)
`cargo clippy`	Lints
`cargo fmt`	Format
`cargo doc --open`	Build HTML docs for your crate and all dependencies
`cargo tree`	Dependency graph; `-e features` shows feature activation
`cargo add serde -F derive`	Add a dependency
`cargo expand`	Show macro-expanded code (install `cargo-expand`)

Worth installing: `cargo-nextest` (faster test runner), `cargo-flamegraph`, `cargo-bloat`, `cargo-udeps`, `cargo-deny` (license and advisory checks).

9.6 Tests live next to the code

```
// src/frame.rs
pub fn checksum(data: &[u8]) -> u32 { crc32fast::hash(data) }

fn pad(n: usize) -> usize { (n + 7) & !7 }        // private

#[cfg(test)]                                       // compiled only for
 ↪ `cargo test`
mod tests {
    use super::*;                                  // includes private
     ↪ items
```

```
    #[test]
    fn pads_to_eight() {
        assert_eq!(pad(1), 8);
        assert_eq!(pad(8), 8);
    }

    #[test]
    #[should_panic(expected = "overflow")]
    fn detects_overflow() { /* ... */ }
}
```

Integration tests go in `tests/*.rs` and see only your public API, like an external user. Doc comments with code blocks are compiled and run as tests, which keeps examples honest:

```
/// Computes the frame checksum.
///
/// ```
/// assert_eq!(kvstore::checksum(b""), 0);
/// ```
pub fn checksum(data: &[u8]) -> u32 { /* ... */ }
```

9.7 Conditional compilation

```
#[cfg(target_os = "linux")]
fn sendfile(out: &File, inp: &File, n: usize) -> io::Result<usize> {
 ↪  /* linux syscall */ }

#[cfg(not(target_os = "linux"))]
fn sendfile(out: &File, inp: &File, n: usize) -> io::Result<usize> {
 ↪  /* read/write loop */ }

#[cfg(feature = "compression")]
pub fn compress(buf: &[u8]) -> Vec<u8> { zstd::encode_all(buf,
 ↪  3).unwrap() }

if cfg!(debug_assertions) { validate_invariants(); }   // expression
 ↪  form
```

Unlike `#ifdef`, `cfg` works on whole items, the code inside must still parse, and features must be additive — enabling a feature should never remove

functionality, because any crate in the graph can turn it on.

9.8 Build scripts

A `build.rs` at the package root runs before compilation. It's where you compile C/C++ sources (`cc` crate), generate bindings (`bindgen`), or embed generated code — the role a custom CMake step plays.

```
// build.rs
fn main() {
    cc::Build::new().cpp(true).file("src/fast_hash.cc").flag("-O3").
    ↪  compile("fasthash");
    println!("cargo:rerun-if-changed=src/fast_hash.cc");
}
```

Remember

- Crate = compilation unit; module = namespace + privacy.
- Items are private by default; `pub(crate)` is the internal-API tool.
- `cargo check` is your inner loop; `clippy` and `fmt` on every change.
- Unit tests sit next to the code and can test private functions; doc examples are tests too.
- Features are additive; `build.rs` handles native code and codegen.

Optional exercises

1. **Workspace.** Create `kv-core` (library), `kv-cli` (binary depending on it), and `kv-bench` (criterion benches). Share dependencies through `[workspace.dependencies]`.
2. **Visibility.** Make a `pub struct Frame` with private fields, a validating `Frame::new`, and a `pub(crate)` helper. Write an integration test in `tests/` and confirm it can't reach the private items.
3. **Feature flag.** Add an optional `compression` feature that pulls in `zstd`. Check with `cargo tree -e features` and build with and without `--features compression`.
4. **Doc tests.** Document three public functions with runnable examples and run `cargo test --doc`. Break one example and read the failure.

5. **Native code.** Use `build.rs` and the `cc` crate to compile a small C++ function into your crate and call it through `extern "C"` (preview of Module 12).

Module 10 — Concurrency

In safe Rust, a data race is a compile error. Two marker traits encode thread safety in types, the standard library's threading APIs require them, and locks own the data they protect. Deadlocks and logic races are still possible; memory-level races are not. About 50 minutes.

10.1 Send and Sync

- `T: Send` — ownership of a `T` may move to another thread.
- `T: Sync` — a `&T` may be shared between threads. Formally, `T: Sync` iff `&T: Send`.

Both are *auto traits*: the compiler implements them for a struct when all its fields have them. You rarely write them, but you constantly see them in signatures and errors.

Type	`Send`	`Sync`	Why
`i32`, `String`, `Vec<T: Send>`	Yes	Yes	Plain owned data
`Rc<T>`	No	No	Non-atomic refcount
`Arc<T: Send + Sync>`	Yes	Yes	Atomic refcount
`Cell<T>`, `RefCell<T>`	Yes (if `T: Send`)	No	Unsynchronized mutation through `&`
`Mutex<T: Send>`	Yes	Yes	Locking makes `&Mutex` safe to share
`MutexGuard<T>`	No	Yes	Must unlock on the locking thread

Type	`Send`	`Sync`	Why
`*const T`, `*mut T`	No	No	Raw pointers opt out; wrap and `unsafe impl` if you've proved safety

The compile-time check in action:

```
use std::rc::Rc;
let data = Rc::new(vec![1, 2, 3]);
std::thread::spawn(move || println!("{:?}", data));
```

```
error[E0277]: `Rc<Vec<i32>>` cannot be sent between threads safely
   = help: within `{closure}`, the trait `Send` is not implemented for
    ↪  `Rc<Vec<i32>>`
note: required by a bound in `spawn`
```

Swap `Rc` for `Arc` and it compiles.

10.2 Threads

```
use std::thread;

let handle = thread::spawn(|| {
    (1..=1_000_000u64).sum::<u64>()
});
let total = handle.join().unwrap();     // Err if the thread panicked
```

`thread::spawn` requires `F: Send + 'static`: the closure can't borrow from the caller's stack, because the thread might outlive it. Move owned data in, or use **scoped threads**, which are guaranteed to finish before the scope ends and can therefore borrow:

```
let data = vec![1u64; 1_000_000];
let (left, right) = data.split_at(data.len() / 2);

let (a, b) = thread::scope(|s| {
    let ha = s.spawn(|| left.iter().sum::<u64>());     // borrows `left`
    let hb = s.spawn(|| right.iter().sum::<u64>());
```

```
    (ha.join().unwrap(), hb.join().unwrap())
});                                                    // all threads
 ↪  joined here
```

That's `std::jthread` plus a compile-time guarantee that borrowed data outlives the threads.

10.3 Shared state: `Mutex<T>` and `RwLock<T>`

```
std::mutex m;
std::unordered_map<std::string, int> counts;   // protected by m — by
 ↪  convention

void bump(const std::string& k) {
    std::lock_guard g(m);
    counts[k]++;
}
```

```
use std::sync::{Arc, Mutex};
use std::collections::HashMap;

let counts = Arc::new(Mutex::new(HashMap::<String, u64>::new()));

let handles: Vec<_> = (0..4).map(|i| {
    let counts = Arc::clone(&counts);
    thread::spawn(move || {
        let mut g = counts.lock().unwrap();     //
         ↪  MutexGuard<HashMap<..>>
        *g.entry(format!("worker-{i}")).or_default() += 1;
    })                                          // guard dropped → unlocked
}).collect();

for h in handles { h.join().unwrap(); }
```

The data is *inside* the mutex. There's no way to reach the `HashMap` without holding the lock, so "forgot to lock" is impossible. `RwLock<T>` gives `read()` and `write()` guards with the same guarantee.

Tips:

- Keep critical sections short; drop the guard early with a block or `drop(g)`.

- `lock()` returns `Err` if another thread panicked while holding the lock (*poisoning*). `unwrap()` is the usual response.
- `parking_lot::Mutex` is smaller, faster under contention, and doesn't poison.

10.4 Message passing

```
use std::sync::mpsc;

let (tx, rx) = mpsc::channel::<Job>();          // or sync_channel(n)
 ↪  for bounded

for id in 0..4 {
    let tx = tx.clone();
    thread::spawn(move || {
        for job in produce(id) { tx.send(job).unwrap(); }
    });
}
drop(tx);                                       // close our copy so rx
 ↪  ends

for job in rx { run(job); }                      // loop ends when all
 ↪  senders drop
```

Sending moves ownership into the channel, so the producer can't touch a job after handing it off. For multi-consumer queues and `select!` over several channels, use `crossbeam-channel` (std's `mpsc` is multi-producer, single-consumer).

10.5 Data parallelism with `rayon`

```
use rayon::prelude::*;

let total: u64 = files.par_iter()                  // was: .iter()
    .map(|f| count_lines(f))
    .sum();

let mut v: Vec<f32> = load();
v.par_sort_unstable_by(|a, b| a.total_cmp(b));
v.par_iter_mut().for_each(|x| *x = x.sqrt());
```

Rayon runs a work-stealing pool, like TBB or the parallel STL. Because closures must be `Send + Sync` where needed, sharing a non-thread-safe value in a parallel closure is a compile error rather than a race.

10.6 Atomics

The memory model is C++11's: same orderings, same semantics.

```
use std::sync::atomic::{AtomicBool, AtomicU64, Ordering};

static REQUESTS: AtomicU64 = AtomicU64::new(0);

fn on_request() {
    REQUESTS.fetch_add(1, Ordering::Relaxed);
}

// Publish data with release/acquire
struct Slot { ready: AtomicBool, value: std::cell::UnsafeCell<u64> }
unsafe impl Sync for Slot {}                    // we promise: writes
 ↪  happen before `ready`

impl Slot {
    fn publish(&self, v: u64) {
        unsafe { *self.value.get() = v; }        // only the single
         ↪  writer does this
        self.ready.store(true, Ordering::Release);
    }
    fn read(&self) -> Option<u64> {
        self.ready.load(Ordering::Acquire).then(|| unsafe {
         ↪  *self.value.get() })
    }
}
```

Rust `Ordering`	C++ `std::memory_order`
`Relaxed`	`memory_order_relaxed`
`Acquire`	`memory_order_acquire`
`Release`	`memory_order_release`
`AcqRel`	`memory_order_acq_rel`
`SeqCst`	`memory_order_seq_cst`

There's no `consume`. Note that the `Slot` example needed `unsafe`: lock-free structures sit on top of `UnsafeCell` and raw pointers, and you uphold the

invariants (Module 12). Test them with `loom`, which runs your test under every legal interleaving and memory-order outcome.

10.7 Other primitives

Need	Rust
Condition variable	`std::sync::Condvar` (paired with a `Mutex`)
Barrier	`std::sync::Barrier`
One-time init	`OnceLock`, `LazyLock`
Thread-local storage	`thread_local!`
Lock-free queues, epoch-based reclamation	`crossbeam` (`queue`, `epoch`, `deque`)
Concurrent hash map	`dashmap`
Semaphore	`tokio::sync::Semaphore` (async) or build from `Mutex` + `Condvar`

10.8 What Rust doesn't catch

- **Deadlocks**: lock ordering is still on you.
- **Logic races**: check-then-act across two separate lock acquisitions.
- **Starvation and priority inversion.**
- **Wrong memory orderings in `unsafe` code**: if you write `Relaxed` where you needed `Acquire`, the compiler can't tell. That's what `loom` and Miri are for.

Remember

- `Send` = may move to another thread; `Sync` = may be shared by reference.
- `thread::spawn` needs `'static` data; `thread::scope` can borrow.
- `Mutex<T>` owns its data; you can't touch it without the guard.
- Channels move ownership between threads; `rayon` parallelizes iterators.
- Atomics follow the C++11 model; lock-free code needs `unsafe` and `loom`.

Optional exercises

1. **Read the error.** Send an `Rc<Vec<i32>>` and a `RefCell<i32>` reference to spawned threads; read both errors. Fix each with the thread-safe counterpart.
2. **Word count three ways.** Count words across a large file: `Arc<Mutex<HashMap>>` with per-thread local maps merged at the end; channels with a single reducer thread; and `rayon` `fold` + `reduce`. Benchmark them.
3. **Scoped parallel sum.** Split a `Vec<u64>` into N chunks and sum them in parallel with `thread::scope`, no `Arc`.
4. **Bounded worker pool.** Build a fixed-size thread pool with a `sync_channel` of jobs (`Box<dyn FnOnce() + Send>`), and graceful shutdown when the sender is dropped.
5. **SPSC ring buffer.** Implement a lock-free single-producer single-consumer queue with `AtomicUsize` head and tail and `Unsafe-Cell<MaybeUninit<T>>` slots. Test it with `loom`. Read Mara Bos's *Rust Atomics and Locks* (free at marabos.nl/atomics) alongside.

Module 11 — Async Rust

An `async fn` compiles to a state machine that implements `Future`, much like a C++20 coroutine. The differences: futures are lazy and poll-driven, their frames are sized at compile time, and the language ships no runtime — you pick one, and for servers that's almost always Tokio. About 50 minutes.

11.1 The basic shape

```rust
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::net::TcpListener;

#[tokio::main]                                  // starts a multithreaded runtime
async fn main() -> anyhow::Result<()> {
    let listener = TcpListener::bind("0.0.0.0:7000").await?;
    loop {
        let (sock, peer) = listener.accept().await?;
        tokio::spawn(async move {               // one lightweight task per connection
            if let Err(e) = handle(sock).await {
                eprintln!("{peer}: {e}");
            }
        });
    }
}

async fn handle(sock: tokio::net::TcpStream) -> std::io::Result<()> {
    let (r, mut w) = sock.into_split();
    let mut lines = BufReader::new(r).lines();
    while let Some(line) = lines.next_line().await? {
        w.write_all(line.to_uppercase().as_bytes()).await?;
        w.write_all(b"\n").await?;
    }
    Ok(())
}
```

This serves tens of thousands of connections on a handful of OS threads. Each `.await` is a potential suspension point where the task yields to the scheduler.

11.2 Futures are lazy

```rust
async fn fetch(id: u32) -> Data { /* ... */ }

let f = fetch(1);           // nothing happens yet: f is just a state
↪  machine value
let d = f.await;            // now it runs, on the current task
```

Calling an `async fn` returns a `Future` without executing any of its body. It runs only when polled — by `.await`, or by handing it to the runtime with `tokio::spawn`. Forgetting to await is a warning (`unused_must_use`), and a silent no-op if ignored.

11.3 Mapping from C++20 coroutines

C++20	Rust
Coroutine frame, usually heap-allocated	`Future` is a plain value with a compile-time size; boxed only when spawned or `Box::pinned`
Eager or lazy, per `promise_type`	Always lazy
`co_await expr`	`expr.await` (postfix, so `fetch().await?.field` chains)
`promise_type`, awaiter protocol	`trait Future { fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<Output>; }`
Resume via `coroutine_handle`	Resume via `Waker::wake()`, which reschedules the task
Executor: bring your own	Runtime crate: `tokio`, `smol`, `embassy` (embedded)
`co_yield` generators	Not stable for async; use the `async-stream` crate or `Stream` trait

11.4 Concurrency inside a task

`tokio::spawn` creates a new task (like a green thread). Within a single task, you can also run several futures concurrently:

```
// Run both, wait for both
let (user, orders) = tokio::join!(fetch_user(id), fetch_orders(id));

// Run both, fail fast on the first error
let (a, b) = tokio::try_join!(load_a(), load_b())?;

// Race: first to finish wins, the others are dropped (cancelled)
tokio::select! {
    res = read_frame(&mut sock) => handle(res?),
    _ = tokio::time::sleep(Duration::from_secs(30)) => return
↪   Err(Timeout),
    _ = shutdown.recv() => return Ok(()),
}

// Many tasks, collect as they complete
let mut set = tokio::task::JoinSet::new();
for url in urls { set.spawn(fetch(url)); }
while let Some(res) = set.join_next().await { process(res??); }
```

11.5 `Send` futures and what you hold across `.await`

On the multithreaded runtime, a spawned task may resume on a different thread after any `.await`, so `tokio::spawn` requires the future to be `Send`. A future is `Send` only if everything it holds across an `.await` is `Send`.

```
async fn bad(state: Arc<std::sync::Mutex<State>>) {
    let guard = state.lock().unwrap();     // std MutexGuard is !Send
    do_io().await;                          // guard held across .await
    guard.update();
}
// tokio::spawn(bad(s));  // error: future cannot be sent between
↪   threads safely
```

Two fixes:

```
// 1. Don't hold the lock across the await (usually best)
async fn good(state: Arc<std::sync::Mutex<State>>) {
    let snapshot = { state.lock().unwrap().snapshot() };   // guard
    ↪   dropped here
```

```
    let result = do_io_with(snapshot).await;
    state.lock().unwrap().apply(result);
}

// 2. Use an async-aware mutex when you truly must hold it while
 ↪  awaiting
async fn also_ok(state: Arc<tokio::sync::Mutex<State>>) {
    let mut g = state.lock().await;
    g.update_with(do_io().await);
}
```

The same applies to `Rc`, `RefCell` borrows, and raw pointers held across an await.

11.6 `Pin`, briefly

An async state machine may hold references into its own frame across a suspension point (a local and a borrow of it). Moving such a value would leave those internal pointers dangling. `Pin<&mut T>` is a pointer that promises the pointee won't move again.

In day-to-day code you meet it only at edges:

```
let fut = some_async_fn();
tokio::pin!(fut);                       // pin on the stack so you can
 ↪  poll it in a loop
loop {
    tokio::select! {
        out = &mut fut => break out,
        _ = tick.tick() => log_progress(),
    }
}

let boxed: Pin<Box<dyn Future<Output = u32> + Send>> = Box::pin(async
 ↪  { 42 });
```

It's the Rust answer to why you can't move a C++ object that has handed out pointers to its members — encoded in the type system.

11.7 Blocking and CPU work

The executor runs many tasks per OS thread. A blocking call (synchronous file I/O, `std::thread::sleep`, a long computation) stalls every task on that worker.

```
// CPU-heavy or blocking: move to the blocking pool
let digest = tokio::task::spawn_blocking(move ||
 ↪  sha256_file(&path)).await??;

// Async-aware alternatives
tokio::time::sleep(Duration::from_millis(10)).await;   // not
 ↪  std::thread::sleep
let bytes = tokio::fs::read("data.bin").await?;
```

For heavy parallel compute, hand work to `rayon` and send results back over a `tokio::sync::oneshot` channel.

11.8 Cancellation is `drop`

Dropping a future cancels it at its last `.await`; no code after that point runs, but destructors do. `select!`, `timeout`, and an aborted `JoinHandle` all cancel by dropping.

```
match tokio::time::timeout(Duration::from_secs(2), fetch(id)).await {
    Ok(data) => use_it(data),
    Err(_elapsed) => metrics.timeouts.inc(),     // fetch() was dropped
     ↪  mid-flight
}
```

Design for *cancel safety*: if an operation is half-done when dropped (bytes read into a local buffer, a message popped from a queue), that state is lost. Tokio's docs mark each method as cancel-safe or not.

For graceful shutdown, pass a `tokio_util::sync::CancellationToken` and `select!` on `token.cancelled()` at safe points.

11.9 Async traits and streams

`async fn` in traits is stable:

```rust
trait Store {
    async fn get(&self, key: &str) -> Option<Vec<u8>>;
}
```

Caveats: such traits aren't usable as `dyn Store` directly, and you can't add a `Send` bound on the returned future without extra syntax. For `dyn` use, the `async-trait` crate (which boxes the future) is still common.

Async iteration uses the `Stream` trait from `futures` / `tokio-stream`: `while let Some(item) = stream.next().await { ... }`.

Remember

- `async fn` returns a lazy `Future`; `.await` or `spawn` drives it.
- Use `join!`/`try_join!`/`select!`/`JoinSet` for concurrency within and across tasks.
- Spawned futures must be `Send`: don't hold `std` mutex guards or `Rc` across `.await`.
- Never block the executor; use `spawn_blocking` or async APIs.
- Cancellation is drop; think about cancel safety at every `select!`.

Optional exercises

1. **Key-value server.** Build a TCP server on Tokio speaking a line protocol (`GET k`, `SET k v`, `DEL k`) with state in `Arc<Mutex<HashMap<String, String>>>`. Load-test it with many concurrent clients.
2. **Timeouts and shutdown.** Add a 5-second idle timeout per connection with `tokio::time::timeout`, and graceful shutdown on Ctrl-C using `tokio::signal::ctrl_c`, `select!`, and a `CancellationToken`.
3. **Hand-written future.** Implement `Future` for a `Delay` that completes after a deadline: in `poll`, if not ready, spawn a thread that sleeps and then calls `waker.wake()`. Drive it with `#[tokio::main]`.
4. **The `Send` error.** Reproduce the `std::sync::MutexGuard`-across-`.await` error inside `tokio::spawn`. Fix it both ways from 11.5 and explain when each is right.
5. **Fan-out.** Fetch 100 URLs (or simulated sleeps) with at most 10 in flight, using `JoinSet` and a `tokio::sync::Semaphore`. Report the p50/p99 latency.

Module 12 — Unsafe Rust and C++ interop

`unsafe` doesn't switch off the borrow checker. It unlocks five extra operations and makes you responsible for invariants the compiler can't verify. The discipline: keep `unsafe` blocks small, wrap them in a safe API, and write down why each one is sound. About 50 minutes.

12.1 What `unsafe` unlocks

Inside an `unsafe` block or `unsafe fn` you may:

1. Dereference raw pointers (`*const T`, `*mut T`).
2. Call `unsafe` functions, including every `extern "C"` function.
3. Implement `unsafe` traits (`Send`, `Sync`, `GlobalAlloc`).
4. Read or write `static mut` variables.
5. Read `union` fields.

Everything else — types, borrows on references, bounds checks — still applies. Creating raw pointers is safe; only dereferencing them is not.

```
let x = 42u32;
let p = &x as *const u32;        // safe: just an address
let y = unsafe { *p };           // unsafe: you assert p is valid and
 ↪  aligned
```

12.2 Rust's UB is different from C++'s

Some C++ UB is defined in Rust; some things fine in C++ are UB in Rust.

Behavior	C++	Rust
Signed integer overflow	UB	Defined: panics (debug) or wraps (release)
Out-of-bounds index	UB	Panic (safe code)

Behavior	C++	Rust
Data race	UB	Impossible in safe code; UB in unsafe
Two live mutable references to the same object	Fine	**UB** — `&mut T` promises exclusivity
Mutating through a `&T` without `UnsafeCell`	Fine with `const_cast` if the object isn't const	**UB**
A `bool` that isn't 0 or 1, an invalid enum discriminant	Unspecified/UB	**UB**, even if never read
Creating a reference to unaligned or uninitialized memory	Often fine	**UB**, even without reading it
Type punning through a `union`	UB (strict aliasing)	Allowed, as long as the bits are valid for the type

The extra rules exist because the compiler optimizes on the guarantees references give (for example, emitting LLVM `noalias`). Raw pointers carry no such guarantees, so inside unsafe code, work with raw pointers as long as possible and create references only when they're valid.

Miri is the tool for checking: an interpreter for Rust's intermediate representation that detects UB, including aliasing violations, as it runs your tests.

```
rustup +nightly component add miri
cargo +nightly miri test
```

12.3 Writing a sound safe abstraction

The pattern: unsafe internals, a safe public API, and invariants documented where they're relied on.

```
use std::mem::MaybeUninit;

/// Fixed-capacity ring buffer.
pub struct Ring<T, const N: usize> {
    buf: [MaybeUninit<T>; N],
    head: usize,      // index of oldest element
    len: usize,       // invariant: elements head..head+len (mod N) are
↪   initialized
```

```
}

impl<T, const N: usize> Ring<T, N> {
    pub fn new() -> Self {
        Self { buf: [const { MaybeUninit::uninit() }; N], head: 0, len:
         ↪  0 }
    }

    pub fn push(&mut self, v: T) -> Result<(), T> {
        if self.len == N { return Err(v); }
        let idx = (self.head + self.len) % N;
        self.buf[idx].write(v);
        self.len += 1;
        Ok(())
    }

    pub fn pop(&mut self) -> Option<T> {
        if self.len == 0 { return None; }
        // SAFETY: len > 0, so buf[head] is initialized by the
         ↪  invariant.
        // We decrement len and advance head, so it won't be read again.
        let v = unsafe { self.buf[self.head].assume_init_read() };
        self.head = (self.head + 1) % N;
        self.len -= 1;
        Some(v)
    }
}

impl<T, const N: usize> Drop for Ring<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}      // drop remaining
         ↪  initialized elements
    }
}
```

Callers can't break the invariant because `head`, `len`, and `buf` are private. That's what makes the `unsafe` block's reasoning local.

Conventions the ecosystem follows:

- Every `unsafe { }` block gets a `// SAFETY:` comment explaining why it's sound.
- Every `pub unsafe fn` gets a `/// # Safety` doc section listing what the caller must guarantee.

- Lint with `#![deny(unsafe_op_in_unsafe_fn)]` (default in edition 2024) and Clippy's `undocumented_unsafe_blocks`.

12.4 Calling C from Rust

```
use std::ffi::{CStr, CString, c_char, c_int};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> usize;
    fn getenv(name: *const c_char) -> *const c_char;
}

pub fn env(name: &str) -> Option<String> {
    let c = CString::new(name).ok()?;                  // adds NUL,
     ↪  rejects interior NULs
    // SAFETY: c is a valid NUL-terminated string for the duration of
     ↪  the call.
    let p = unsafe { getenv(c.as_ptr()) };
    if p.is_null() { return None; }
    // SAFETY: getenv returned a non-null, NUL-terminated string owned
     ↪  by libc.
    Some(unsafe { CStr::from_ptr(p) }.to_string_lossy().into_owned())
}
```

Shared structs use `#[repr(C)]` so the layout matches C:

```
#[repr(C)]
pub struct Header { magic: u32, version: u16, flags: u16, len: u64 }
```

Wrap C resources in RAII types:

```
pub struct Uring { raw: *mut sys::io_uring }
impl Uring {
    pub fn new(entries: u32) -> io::Result<Self> {
        let raw = Box::into_raw(Box::new(unsafe {
         ↪  std::mem::zeroed::<sys::io_uring>() }));
        // SAFETY: raw points to writable, properly sized memory.
        let rc = unsafe { sys::io_uring_queue_init(entries, raw, 0) };
        if rc < 0 {
            drop(unsafe { Box::from_raw(raw) });
            return Err(io::Error::from_raw_os_error(-rc));
        }
        Ok(Self { raw })
```

```
    }
}
impl Drop for Uring {
    fn drop(&mut self) {
        // SAFETY: raw was initialized in new() and is freed exactly
        ↪ once here.
        unsafe { sys::io_uring_queue_exit(self.raw);
        ↪ drop(Box::from_raw(self.raw)); }
    }
}
```

12.5 Calling Rust from C or C++

```
#[unsafe(no_mangle)]
pub extern "C" fn kv_checksum(data: *const u8, len: usize) -> u32 {
    if data.is_null() { return 0; }
    // SAFETY: caller guarantees data points to len readable bytes.
    let bytes = unsafe { std::slice::from_raw_parts(data, len) };
    crc32fast::hash(bytes)
}
```

Set `crate-type = ["staticlib"]` or `["cdylib"]` in `Cargo.toml`, generate a header with `cbindgen`, and link it like any C library.

12.6 Real C++ interop with cxx

C ABI bridges lose C++ types. The cxx crate generates both sides of a typed bridge and understands `std::string`, `std::vector`, `std::unique_ptr`, and Rust's `String`, `Vec`, `Box`, and slices.

```
// src/lib.rs
#[cxx::bridge(namespace = "engine")]
mod ffi {
    struct Stats { rows: u64, bytes: u64 }          // shared struct,
    ↪ same layout both sides

    unsafe extern "C++" {
        include!("engine/include/scanner.h");
        type Scanner;                                // opaque C++ type
        fn new_scanner(path: &str) -> UniquePtr<Scanner>;
```

```
        fn next_batch(self: Pin<&mut Scanner>, out: &mut Vec<u8>) ->
        ↪ bool;
        fn stats(&self) -> Stats;
    }

    extern "Rust" {
        fn checksum(data: &[u8]) -> u32;           // callable from C++
    }
}

fn checksum(data: &[u8]) -> u32 { crc32fast::hash(data) }

pub fn scan_all(path: &str) -> ffi::Stats {
    let mut s = ffi::new_scanner(path);
    let mut buf = Vec::new();
    while s.pin_mut().next_batch(&mut buf) { buf.clear(); }
    s.stats()
}
```

```
// build.rs
fn main() {
    cxx_build::bridge("src/lib.rs")
        .file("engine/src/scanner.cc")
        .std("c++20")
        .compile("engine");
}
```

Tool	Direction	Use when
`extern "C"` + `#[repr(C)]`	Both	A small C ABI surface you control
`bindgen`	C/C++ headers → Rust	Wrapping an existing C library (liburing, libibverbs)
`cbindgen`	Rust → C header	Shipping a Rust library to C/C++ users
`cxx`	Both, C++ types	Incrementally mixing Rust into a C++ codebase
`autocxx`	C++ → Rust	Larger C++ APIs, generated on top of `cxx`

12.7 Panics and exceptions at the boundary

A Rust panic must not unwind into C/C++ frames, and a C++ exception must not unwind into Rust frames. With `extern "C"`, a panic that reaches the boundary aborts the process. To handle it, catch it first:

```
#[unsafe(no_mangle)]
pub extern "C" fn kv_process(p: *const u8, n: usize) -> c_int {
    match std::panic::catch_unwind(|| process(unsafe {
     ↪  std::slice::from_raw_parts(p, n) })) {
        Ok(Ok(())) => 0,
        Ok(Err(_)) => -1,
        Err(_panic) => -2,
    }
}
```

`cxx` converts C++ exceptions into Rust `Results` for functions declared to return `Result<T>`.

Remember

- `unsafe` unlocks five operations; all other rules still apply.
- Rust's UB list differs from C++'s: aliasing `&mut` and invalid values are UB.
- Wrap unsafe internals in a safe API with private fields; document every `SAFETY:` argument.
- Run Miri on code with `unsafe`.
- `bindgen` and `cbindgen` for C ABIs, `cxx` for typed C++ bridges; never unwind across the boundary.

Optional exercises

1. **Ring buffer under Miri.** Type in `Ring<T, N>` from 12.3 and write tests with a `Drop`-counting element type. Run `cargo +nightly miri test`. Then introduce a bug (forget to decrement `len`) and see what Miri reports.
2. **Spot the UB.** Write a function that creates two `&mut` to the same `i32` via raw pointers and writes through both. Run it under Miri. Rewrite it using only raw pointers until Miri is happy.

3. **`cxx` bridge.** Wrap a small C++ class you know (a CRC32, a bloom filter, a column vector) in a `cxx` bridge, and call a Rust function from C++. Build everything through Cargo.
4. **`bindgen` + RAII.** Generate bindings for `liburing` or `libibverbs` with `bindgen` and wrap opening a device or ring in a type with `Drop`. Expose one safe operation.
5. **Panic boundary.** Export an `extern "C"` function that can panic, call it from a C program, and observe the abort. Add `catch_unwind` and return an error code instead.

Module 13 — Macros

Rust macros work on token trees, not text, and they're hygienic — no `#define` surprises, no accidental name capture. Most of what C++ does with the preprocessor or template metaprogramming, Rust does with generics, `const fn`, const generics, and `cfg`. Macros are for syntax-level repetition and code generation. About 35 minutes.

13.1 Recognizing macros

A macro call ends in `!`: `println!`, `vec!`, `format!`, `assert_eq!`, `matches!`, `write!`. Attributes like `#[derive(Debug)]` and `#[tokio::main]` are also macros. They exist where a function can't do the job — variable arguments, compile-time format-string checking, or generating items.

```
let v = vec![0u8; 4096];                          // expands to a Vec
 ↪  allocation
println!("{name} has {} items", v.len());       // format string
 ↪  checked at compile time
assert!(matches!(state, State::Open { .. }));  // pattern as an
 ↪  expression
let path = concat!(env!("OUT_DIR"), "/gen.rs");  // compile-time
 ↪  string building
```

13.2 Compile-time computation without macros

Before reaching for a macro, check whether these cover it:

```
// const fn: evaluated at compile time when used in a const context
 ↪  (like constexpr)
const fn kib(n: usize) -> usize { n * 1024 }
const PAGE: usize = kib(4);

// const generics: like non-type template parameters
struct Matrix<const R: usize, const C: usize> { data: [[f32; C]; R] }
```

```
impl<const R: usize, const C: usize> Matrix<R, C> {
    fn transpose(&self) -> Matrix<C, R> {
        let mut out = Matrix { data: [[0.0; R]; C] };
        for i in 0..R { for j in 0..C { out.data[j][i] =
        ↪ self.data[i][j]; } }
        out
    }
}

// compile-time assertions
const _: () = assert!(std::mem::size_of::<Header>() == 16);
```

C++ technique	Rust tool
`constexpr` function	`const fn`
`static_assert`	`const _: () = assert!(..);`
Non-type template parameter	Const generic `<const N: usize>`
`#ifdef` / `#if`	`#[cfg(..)]`, `cfg!(..)`
X-macros, token pasting	`macro_rules!`
Code generators, reflection-style boilerplate	Procedural macros

13.3 Declarative macros: `macro_rules!`

A declarative macro matches its input against patterns of tokens and expands the first one that fits.

```
macro_rules! hashmap {
    () => { ::std::collections::HashMap::new() };
    ($($k:expr => $v:expr),+ $(,)?) => {{
        let mut m = ::std::collections::HashMap::new();
        $( m.insert($k, $v); )+
        m
    }};
}

let ports = hashmap! { "http" => 80, "https" => 443, };
```

The pieces:

- `$k:expr` captures an expression into `$k`. Other *fragment specifiers*: `ident`, `ty`, `pat`, `path`, `literal`, `block`, `tt` (any single token tree), `item`, `lifetime`, `vis`.

- `$( ... ),+` repeats one or more times separated by commas; `*` is zero or more; `?` is optional. The same repetition in the output emits each captured item.
- `::std::...` uses absolute paths so the macro works wherever it's called, even if the caller has shadowed `std`.

A more practical example — generating newtype ids, the kind of boilerplate an X-macro would produce in C++:

```
macro_rules! id_types {
    ($($name:ident),* $(,)?) => {
        $(
            #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash,
             ↪ PartialOrd, Ord)]
            pub struct $name(pub u32);

            impl std::fmt::Display for $name {
                fn fmt(&self, f: &mut std::fmt::Formatter<'_>) ->
                 ↪ std::fmt::Result {
                    write!(f, "{}#{}", stringify!($name), self.0)
                }
            }
        )*
    };
}

id_types!(TableId, ColumnId, SegmentId);
println!("{}", TableId(7));    // TableId#7
```

Hygiene. Local variables a macro introduces can't clash with the caller's:

```
macro_rules! double { ($e:expr) => {{ let x = $e; x * 2 }} }
let x = 10;
let y = double!(x + 1);    // 22: the macro's `x` and yours are
 ↪ different bindings
```

The C preprocessor equivalent would either capture `x` or need `(e)` parentheses to get precedence right. Rust macros receive `$e` as a single parsed expression, so precedence is never an issue.

Scoping: a `macro_rules!` is usable after its definition in the same module. Export it from a crate with `#[macro_export]`, and users import it with `use mycrate::hashmap;`.

13.4 Procedural macros

Procedural macros are Rust functions that run inside the compiler, taking a `TokenStream` and returning a new one. They live in a separate crate with `proc-macro = true`, and almost always use `syn` (parse Rust syntax) and `quote` (generate it). Three kinds:

Kind	Looks like	Examples
Derive	`#[derive(Serialize)]`	`serde`, `thiserror`, `clap::Parser`
Attribute	`#[tokio::main]`, `#[instrument]`	`tokio`, `tracing`, `async-trait`
Function-like	`sql!(SELECT ...)`	`sqlx::query!`, `html!`

A small derive that adds a method listing a struct's field names:

```rust
// field_names/src/lib.rs   (Cargo.toml: [lib] proc-macro = true)
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, Data, DeriveInput, Fields};

#[proc_macro_derive(FieldNames)]
pub fn derive_field_names(input: TokenStream) -> TokenStream {
    let ast = parse_macro_input!(input as DeriveInput);
    let name = &ast.ident;
    let fields: Vec<String> = match &ast.data {
        Data::Struct(s) => match &s.fields {
            Fields::Named(f) => f.named.iter().map(|f|
            ↪  f.ident.as_ref().unwrap().to_string()).collect(),
            _ => vec![],
        },
        _ => panic!("FieldNames only supports structs"),
    };
    quote! {
        impl #name {
            pub const FIELD_NAMES: &'static [&'static str] =
            ↪  &[#(#fields),*];
        }
    }.into()
}
```

```
// user crate
#[derive(field_names::FieldNames)]
struct Row { id: u64, name: String, score: f32 }

assert_eq!(Row::FIELD_NAMES, ["id", "name", "score"]);
```

`quote!` uses `#var` for interpolation and `#(#items),*` for repetition — the same idea as `macro_rules!` but with ordinary Rust code driving it. For real crates, report errors with `syn::Error::to_compile_error()` instead of panicking, so the user sees a pointed message at the right span.

Procedural macros add compile time (they're compiled and run before your crate) and are harder to debug, so use them for boilerplate that repeats across many types.

13.5 Debugging macros

- `cargo expand` (install with `cargo install cargo-expand`) prints your code after macro expansion — the equivalent of `gcc -E`, but readable.
- rust-analyzer can expand a macro at the cursor ("Expand macro recursively").
- For `macro_rules!`, build up one rule at a time and test each with a small call.

Remember

- Prefer `const fn`, const generics, and `cfg` before macros.
- `macro_rules!` matches token patterns; it's hygienic and precedence-safe.
- Use `$crate::` / `::std::` paths in exported macros.
- Procedural macros (derive, attribute, function-like) are compiler plugins built with `syn` and `quote`.
- `cargo expand` shows what a macro produced.

Optional exercises

1. **`hashmap!` and `hashset!`.** Type in `hashmap!`, add a `hashset!`, and give both a `with_capacity` rule that pre-sizes from the number of entries

(hint: count repetitions with a helper macro).

2. **Bitflags-lite.** Write `macro_rules! flags` that generates a `u32` newtype with named constants, `|` and `&` operators, and `contains()`. Inspect the expansion with `cargo expand`.
3. **FieldNames derive.** Build the `FieldNames` derive from 13.4 as a proc-macro crate in a workspace. Make it return a proper compile error for enums.
4. **Const generics.** Implement `ArrayVec<T, const N: usize>` with `push`, `pop`, `len`, and `Deref<Target = [T]>`, using `MaybeUninit` (combines Modules 6, 12, and 13).
5. **Compile-time checks.** Add `const _: () = assert!(...)` checks for the size and alignment of a `#[repr(C)]` wire header, then change a field type and see the build fail.

Appendix A — Capstone projects

Pick one project and take it to something you would show in a code review: tests, benchmarks, Clippy-clean, documented `unsafe`. Each one forces a different set of modules to work together.

Project	Exercises modules	Scope (approx.)	What makes it hard
Columnar query kernel — Arrow-style column vectors, filter/project/hash-aggregate operators over `&[T]` slices, SIMD via std::arch intrinsics, nightly `std::simd` or `wide`	3, 6, 8, 12	2–3 weeks	Generic kernels over many types without code bloat; zero-copy borrowing of batches
LSM-tree key-value store — memtable, WAL, SSTables, compaction thread, `mmap` reads	2, 3, 7, 10, 12	3–4 weeks	Sharing immutable SSTables across threads with `Arc`; crash recovery; safe `mmap` wrappers
Raft-replicated KV service on Tokio — leader election, log replication, gRPC via `tonic`	5, 6, 10, 11	3–4 weeks	Cancellation-safe async state machines; deterministic testing (look at `turmoil` or `madsim`)

Project	Exercises modules	Scope (approx.)	What makes it hard
Parallel data loader — reads Parquet with the `parquet`/`arrow` crates, decodes and batches on a `rayon` pool, exposes a Python iterator via `PyO3`	7, 8, 10, 12	2–3 weeks	Backpressure between stages; GIL and FFI boundaries; zero-copy hand-off to Python
Rust ↔ C++ bridge — port one self-contained component of a C++ codebase you know into Rust behind a `cxx` bridge, keeping its tests green	2, 6, 9, 12	2 weeks	The real-world migration skill; API shape across the boundary

The data-loader project suits readers coming from data-processing or ML infrastructure, and the bridge project is the most direct path to using Rust inside an existing C++ codebase.

Done means

- ☐ `cargo clippy -- -D warnings` passes
- ☐ Unit and integration tests, plus at least one property test (`proptest`)
- ☐ `criterion` benchmarks with a C++ baseline for comparison where one exists
- ☐ Every `unsafe` block has a `SAFETY:` comment, and Miri passes on the test suite
- ☐ Concurrency code tested under `loom` or a deterministic simulator
- ☐ Public API documented with runnable examples

Appendix B — C++ → Rust cheat sheet

A quick lookup for the idioms you will reach for most; the modules explain the differences.

C++	Rust
`const T&`	`&T`
`T&`	`&mut T`
`T*` (nullable, non-owning)	`Option<&T>` or `*const T` in unsafe code
`std::move(x)`	`x` (moves are the default)
Copy constructor	`.clone()` / `#[derive(Clone)]`
Destructor	`impl Drop`
`auto x = ...;`	`let x = ...;` (immutable) / `let mut x`
`constexpr`	`const`, `const fn`
`static` local / global	`static`, `OnceLock`, `LazyLock`
`namespace`	`mod`
`#include` / `using`	`use`
`std::string` / `std::string_view`	`String` / `&str`
`std::vector<T>` / `std::span<T>`	`Vec<T>` / `&[T]`
`std::array<T, N>`	`[T; N]`
`std::pair` / `std::tuple`	`(A, B)`
`std::optional<T>`	`Option<T>`
`std::expected<T, E>` / exceptions	`Result<T, E>` + `?`
`std::variant`	`enum`
`std::function<R(A)>`	`Box<dyn Fn(A) -> R>`
Lambda `[&]` / `[=]`	Closure; a `move` closure captures by value
`template<typename T>`	`fn f<T: Bound>()`
`virtual`	`dyn Trait`
`static_cast<int>(x)`	`x as i32` (lossy) or `i32::try_from(x)?`
`reinterpret_cast`	`std::mem::transmute` (unsafe; avoid)
`assert` / `static_assert`	`assert!`, `debug_assert!` / `const _: () = assert!(...)`
`printf` / `std::format`	`println!("{x} {y:?}")`
`for (auto& x : v)`	`for x in &v`

C++	Rust
`for (size_t i = 0; i < n; ++i)`	`for i in 0..n`
`switch`	`match` (exhaustive)
`nullptr` checks	`if let Some(x) = opt`
CMake / Conan / vcpkg	Cargo / crates.io
gtest / Google Benchmark	`#[test]` / `criterion`
ASan / UBSan / TSan	Mostly unnecessary in safe code; Miri and `loom` for the rest
`perf`, `valgrind`	Same tools work; plus `cargo flamegraph`

Appendix C — Resources

Ordered by when to use them in the course; the first four are free and online.

Resource	When	Why
The Rust Programming Language (“the Book”)	Weeks 1–4, alongside the modules	The canonical reference; skim what you know
Rust by Example	Any time	Runnable snippets per feature
Rustlings	Weeks 1–3, 20 minutes a day	Small compile-fail exercises that drill the borrow checker
The Rustonomicon	Module 12	Unsafe Rust, aliasing, layout, FFI
Rust Atomics and Locks (Mara Bos)	Module 10	Memory model and lock-free code, written for systems programmers; free online
Programming Rust, 2nd ed. (Blandy, Orendorff, Tindall)	Throughout	Best book-length treatment for experienced C/C++ developers
Rust for Rustaceans (Jon Gjengset)	After week 4	Intermediate idioms, API design, unsafe, async internals
Async book and Tokio tutorial	Module 11	Futures, `Pin`, runtimes
cxx.rs	Module 12	Safe C++ interop
Clippy lint list	Throughout	Explains why each idiom is preferred
Compiler Explorer	Throughout	Compare Rust and C++ codegen side by side
Jon Gjengset’s “Crust of Rust” videos	Weeks 3–6	Live-coded deep dives on lifetimes, iterators, smart pointers, atomics